

Chapter 8

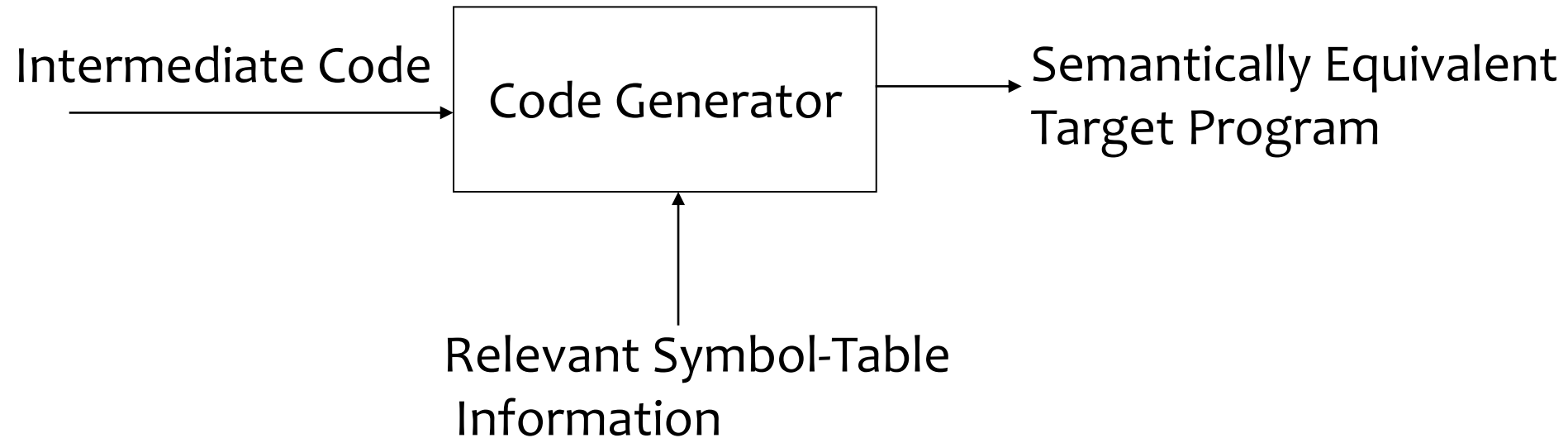
Code Generation

Code Generation

- Final phase of our compiler model

Code Generation

- Final phase of our compiler model



Important Issues of Code Generation

Important Issues of Code Generation

- Should produce correct Target Program
 - Preserving Semantic meaning of Source Program
- Should produce high quality Target Program
 - Efficient use of available resources of target program
- Should run efficiently

Code Generation & Optimization

Code Generation & Optimization

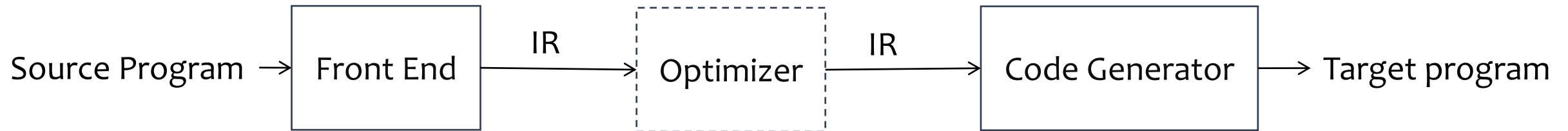
- To produce optimized target program, compiler may include an optimization phase prior to the generation
- Optimizer maps the IR into IR from which efficient code can be generated

Code Generation & Optimization

- To produce optimized target program, compiler may include an optimization phase prior to the generation
- Optimizer maps the IR into IR from which efficient code can be generated

Code Generation & Optimization

- To produce optimized target program, compiler may include an optimization phase prior to the generation
- Optimizer maps the IR into IR from which efficient code can be generated



Position of a Code Generator

Issue 1: Input to Code Generator

Issue 1: Input to Code Generator

- The intermediate representation (IR) of the source program.
 - Three address representation
 - Quadruples
 - Triples
 - Indirect triples
 - Virtual machine representation
 - Byte codes and stack-machine code
 - Linear representation
 - Postfix notations
 - Graphical representation
 - Syntax tree and DAG
- We consider Three address representation.

Issue 1: Input to Code Generator

Issue 1: Input to Code Generator

- Code generator assumes that
 - Front end has scanned, parsed, translated the source program into a relatively low level IR
 - All syntactic and static semantic errors have been detected
 - The necessary type checking has taken place
 - Type conversion operators have been inserted

Issue 2: Target Machine and Program

Issue 2: Target Machine and Program

- Instruction set architecture of the target machine has significant impact on code generation
- The most common machine architectures are:

➤ RISC

- Many Registers
- Three address instructions
- Simple addressing modes
- Relatively simple instruction – set architecture

➤ CISC

- Few registers
- Two address instructions
- Variety of addressing modes
- Variable length instructions

➤ Stack Based

- For example, JVM

Issue 2: Target Machine and Program (2)

Issue 2: Target Machine and Program (2)

- Absolute machine code
 - It can be placed in a fixed location in memory
 - Can immediately be executed
- Relocatable machine – language program
 - Allows subprograms to be compiled separately
 - Linker : links a set of Relocatable object modules together
 - Loader: load them in memory
- Assembly language
 - Easier to generate using of macro and symbolic instruction
 - However, requires assembly step after code generation

Issue 3: Instruction Selection

Issue 3: Instruction Selection

- The code generator must map the IR program into a code sequence that can be executed by the machine
- The complexity of performing the mapping is determined by factors such as
 - The level of the IR
 - The nature of the ISA (instruction-set architecture)
 - The desired quality of the generated code
- From a set of instructions, we need to select instructions such that
 - Generated code is correct and efficient
- **Instruction selection is easy if we don't care efficiency**
 - For each three address statement, we can design a code skeleton that defines the target code for that construct

Issue 3: Instruction Selection

Issue 3: Instruction Selection

```
LD  R0, y      // R0 = y      (load y into register R0)
ADD R0, R0, z   // R0 = R0 + z (add z to R0)
ST  x, R0      // x = R0      (store R0 into x)
```

Issue 3: Instruction Selection

- Instruction selection for three address statement of form $x = y + z$ can be:

```
LD  R0, y      // R0 = y      (load y into register R0)
ADD R0, R0, z   // R0 = R0 + z (add z to R0)
ST  x, R0      // x = R0      (store R0 into x)
```

- But can produce redundant loads and stores

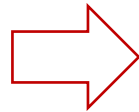
Issue 3: Instruction Selection

- Instruction selection for three address statement of form $x = y + z$ can be:

```
LD  R0, y      // R0 = y      (load y into register R0)
ADD R0, R0, z   // R0 = R0 + z (add z to R0)
ST  x, R0      // x = R0      (store R0 into x)
```

- But can produce redundant loads and stores

```
a = b + c
d = a + e
```



Three address statements

Translated Code

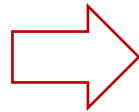
Issue 3: Instruction Selection

- Instruction selection for three address statement of form $x = y + z$ can be:

```
LD  R0, y      // R0 = y      (load y into register R0)
ADD R0, R0, z   // R0 = R0 + z (add z to R0)
ST  x, R0      // x = R0      (store R0 into x)
```

- But can produce redundant loads and stores

```
a = b + c
d = a + e
```



```
LD  R0, b      // R0 = b
ADD R0, R0, c   // R0 = R0 + c
ST  a, R0      // a = R0
```

Three address statements

Translated Code

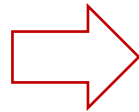
Issue 3: Instruction Selection

- Instruction selection for three address statement of form $x = y + z$ can be:

```
LD  R0, y      // R0 = y      (load y into register R0)
ADD R0, R0, z   // R0 = R0 + z (add z to R0)
ST  x, R0      // x = R0      (store R0 into x)
```

- But can produce redundant loads and stores

```
a = b + c
d = a + e
```



```
LD  R0, b      // R0 = b
ADD R0, R0, c   // R0 = R0 + c
ST  a, R0      // a = R0
LD  R0, a      // R0 = a
ADD R0, R0, e   // R0 = R0 + e
ST  d, R0      // d = R0
```

Three address statements

Translated Code

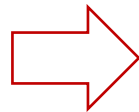
Issue 3: Instruction Selection

- Instruction selection for three address statement of form $x = y + z$ can be:

```
LD  R0, y      // R0 = y      (load y into register R0)
ADD R0, R0, z   // R0 = R0 + z (add z to R0)
ST  x, R0      // x = R0      (store R0 into x)
```

- But can produce redundant loads and stores

```
a = b + c
d = a + e
```



```
LD  R0, b      // R0 = b
ADD R0, R0, c   // R0 = R0 + c
ST  a, R0      // a = R0
LD  R0, a      // R0 = a
ADD R0, R0, e   // R0 = R0 + e
ST  d, R0      // d = R0
```

Three address statements

Translated Code

Issue 3: Instruction Selection

```
LD  R0, a      // R0 = a
ADD R0, R0, #1  // R0 = R0 + 1
ST  a, R0      // a = R0
```

(i)

Issue 3: Instruction Selection

- On most machines, a given IR can be translated in many different ways
 - Some may be efficient, some not
- Example statement: $a = a + 1$
 - If the target machine has an INC instruction, then we have two ways to implement for that machine

```
LD  R0, a      // R0 = a
ADD R0, R0, #1  // R0 = R0 + 1
ST  a, R0      // a = R0
```

(i)

```
INC a
```

(ii)

Issue 4: Register Allocation

Issue 4: Register Allocation

- Registers are the fastest computational unit but we do not have enough of them
- Values not held in register need to reside in memory
- Instruction involving register operands are shorter and faster
- Use of registers is subdivided into two subproblems –
 - **Register allocation**: select the set of variables that will reside in registers at each point in the program
 - **Register assignment**: we pick a register that a variable will reside in.
- Finding an optimal assignment of registers to variables is difficult
 - **NP-complete**

Issue 5: Evaluation Order

Issue 5: Evaluation Order

- Evaluation order can affect the efficiency of the target code
 - Some ordering requires fewer registers to hold intermediate results
- Finding the best order is difficult
 - NP-complete

Basic Blocks & Flow Graphs

Basic Blocks & Flow Graphs

- *Flow graph* is a graphical representation of Intermediate code (IR)
 - Helps in Code Generation
 - Better job in Register Allocation
 - Better job in Instruction Selection

Basic Blocks & Flow Graphs

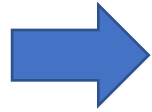
```
for  $i$  from 1 to 10 do  
    for  $j$  from 1 to 10 do  
         $a[i, j] = 0.0;$   
for  $i$  from 1 to 10 do  
     $a[i, i] = 1.0;$ 
```

Source Program

Basic Blocks & Flow Graphs

```
for i from 1 to 10 do
    for j from 1 to 10 do
        a[i, j] = 0.0;
for i from 1 to 10 do
    a[i, i] = 1.0;
```

Source Program



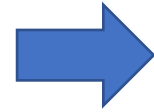
```
1)  i = 1
2)  j = 1
3)  t1 = 10 * i
4)  t2 = t1 + j
5)  t3 = 8 * t2
6)  t4 = t3 - 88
7)  a[t4] = 0.0
8)  j = j + 1
9)  if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

Three Address Code

Basic Blocks & Flow Graphs

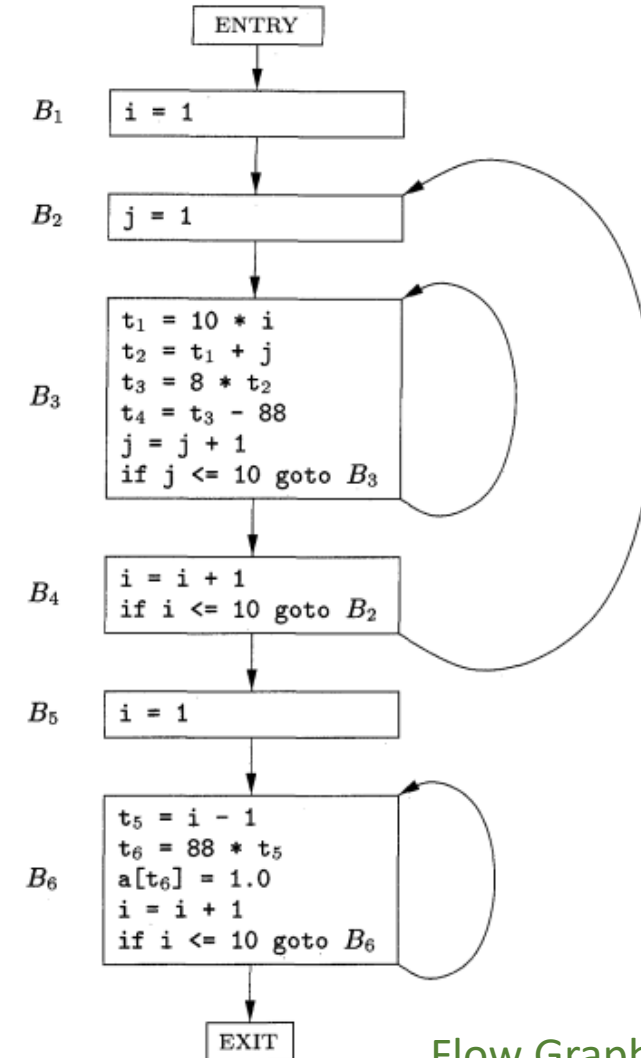
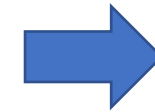
```
for i from 1 to 10 do
    for j from 1 to 10 do
        a[i, j] = 0.0;
for i from 1 to 10 do
    a[i, i] = 1.0;
```

Source Program



```
1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

Three Address Code



Flow Graph

Constructing Flow Graphs

Constructing Flow Graphs

- Partition the intermediate code into *basic blocks*, which are *maximal sequences of consecutive Three-Address instructions* with the properties –
 - The flow of control can only enter the basic block through the first instruction in the block
 - There are no jumps into the middle of the block
 - Control will leave the block without halting or branching
 - Except possibly at the last instruction in the block
- The basic blocks become the *nodes* of a *flow graph*
 - *Edges* indicate which block can follow which other blocks

Constructing Basic Blocks

Constructing Basic Blocks

- **Algorithm:** Partitioning Three-Address Code into Basic Blocks
- **Input:** A sequence of Three-Address Code
- **Output:** A **list of basic blocks** where each instruction is assigned to exactly one block
- **Method:**
 1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
 2. For each leader, basic block consists codes upto next leader or end

Constructing Basic Blocks

Method:

1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end

```
1)  i = 1
2)  j = 1
3)  t1 = 10 * i
4)  t2 = t1 + j
5)  t3 = 8 * t2
6)  t4 = t3 - 88
7)  a[t4] = 0.0
8)  j = j + 1
9)  if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

Constructing Basic Blocks

Method:

1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end

```
1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

Leader (Rule 1) →

Constructing Basic Blocks

Method:

1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end

```
1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

Leader (Rule 1) points to instruction 1)

Leader (Rule 2) points to instruction 2)

Constructing Basic Blocks

Method:

1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end

```
1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

Leader (Rule 1) points to instruction 1)

Leader (Rule 2) points to instruction 2)

Leader (Rule 2) points to instruction 3)

Constructing Basic Blocks

Method:

1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end

```
1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

Leader (Rule 1) points to instruction 1)

Leader (Rule 2) points to instruction 2)

Leader (Rule 2) points to instruction 3)

Leader (Rule 3) points to instruction 10)

Constructing Basic Blocks

Method:

1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end

```
1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

Leader (Rule 1) points to instruction 1)

Leader (Rule 2) points to instruction 2)

Leader (Rule 2) points to instruction 3)

Leader (Rule 3) points to instruction 10)

Leader (Rule 3) points to instruction 12)

Constructing Basic Blocks

Method:

1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end

```
1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

Leader (Rule 1) points to instruction 1)

Leader (Rule 2) points to instruction 2)

Leader (Rule 2) points to instruction 3)

Leader (Rule 3) points to instruction 10)

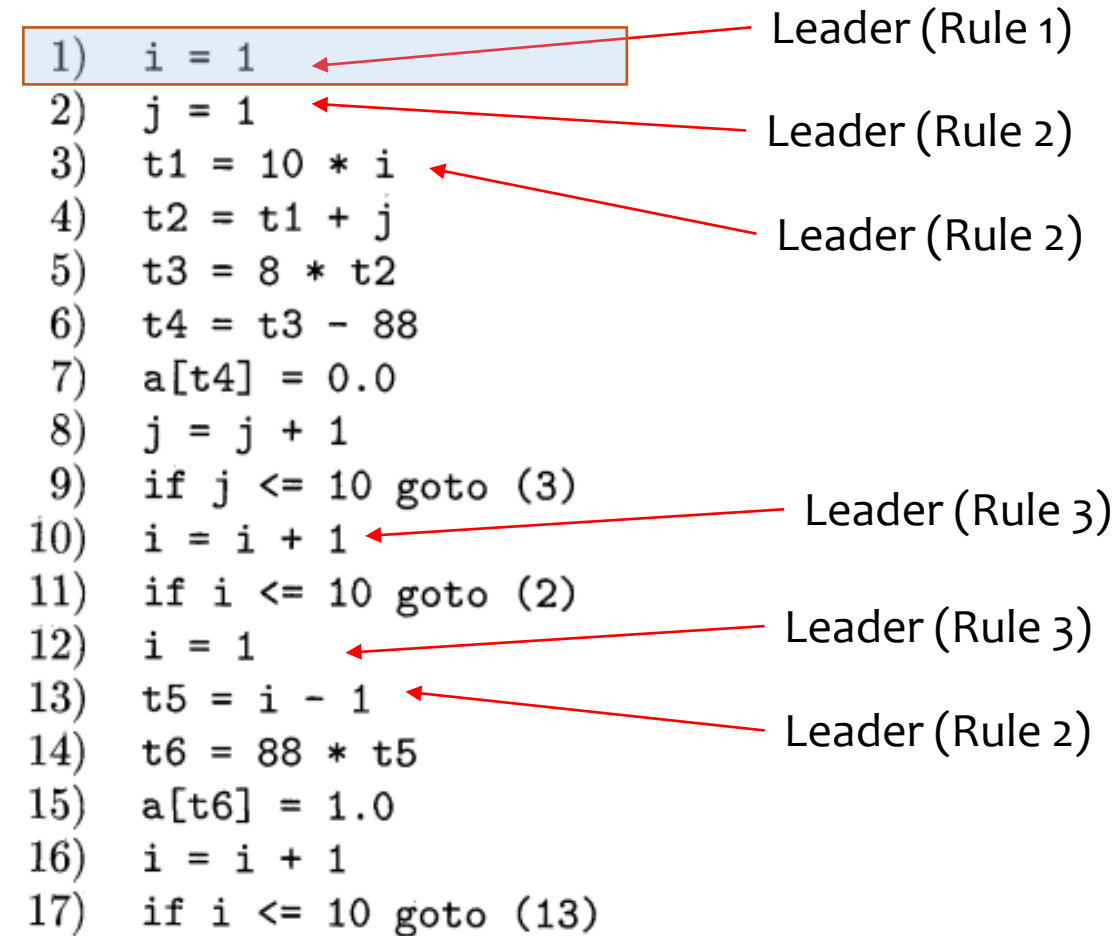
Leader (Rule 3) points to instruction 12)

Leader (Rule 2) points to instruction 13)

Constructing Basic Blocks

Method:

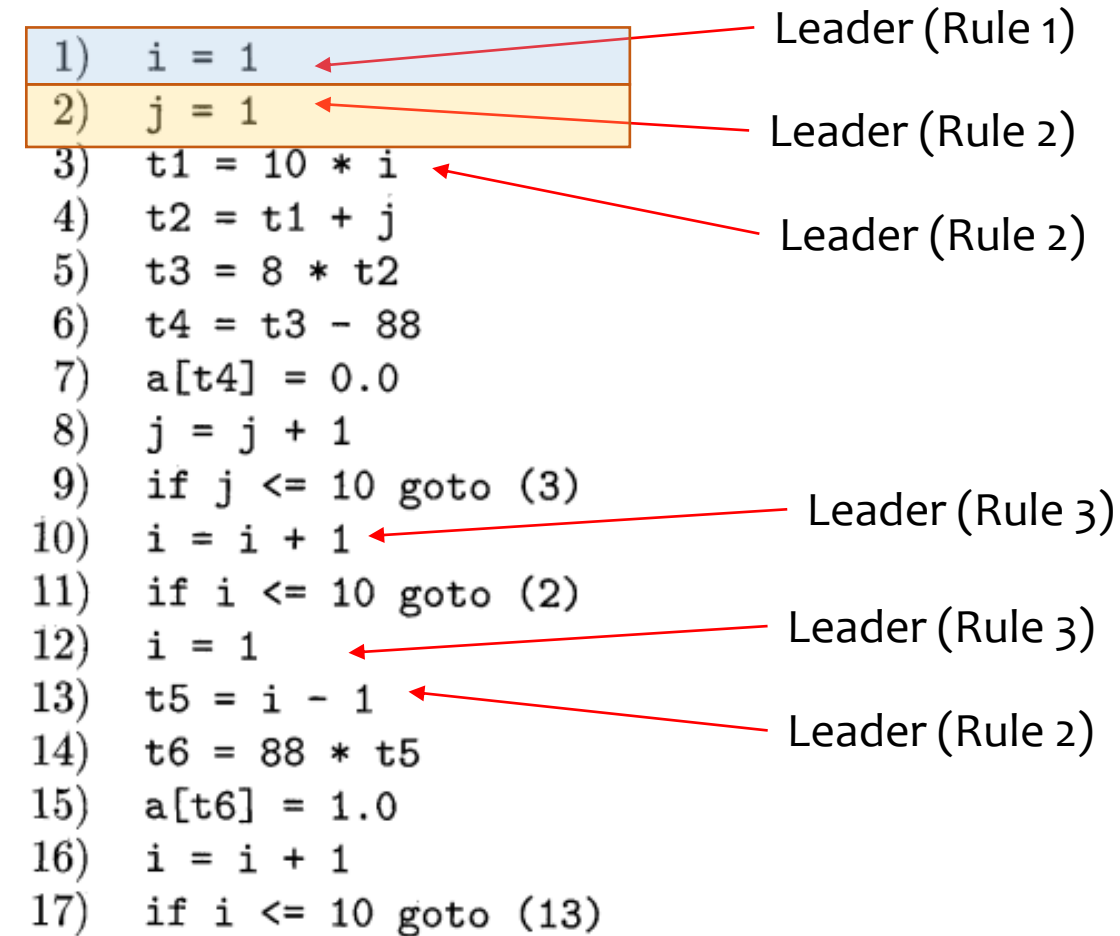
1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end



Constructing Basic Blocks

Method:

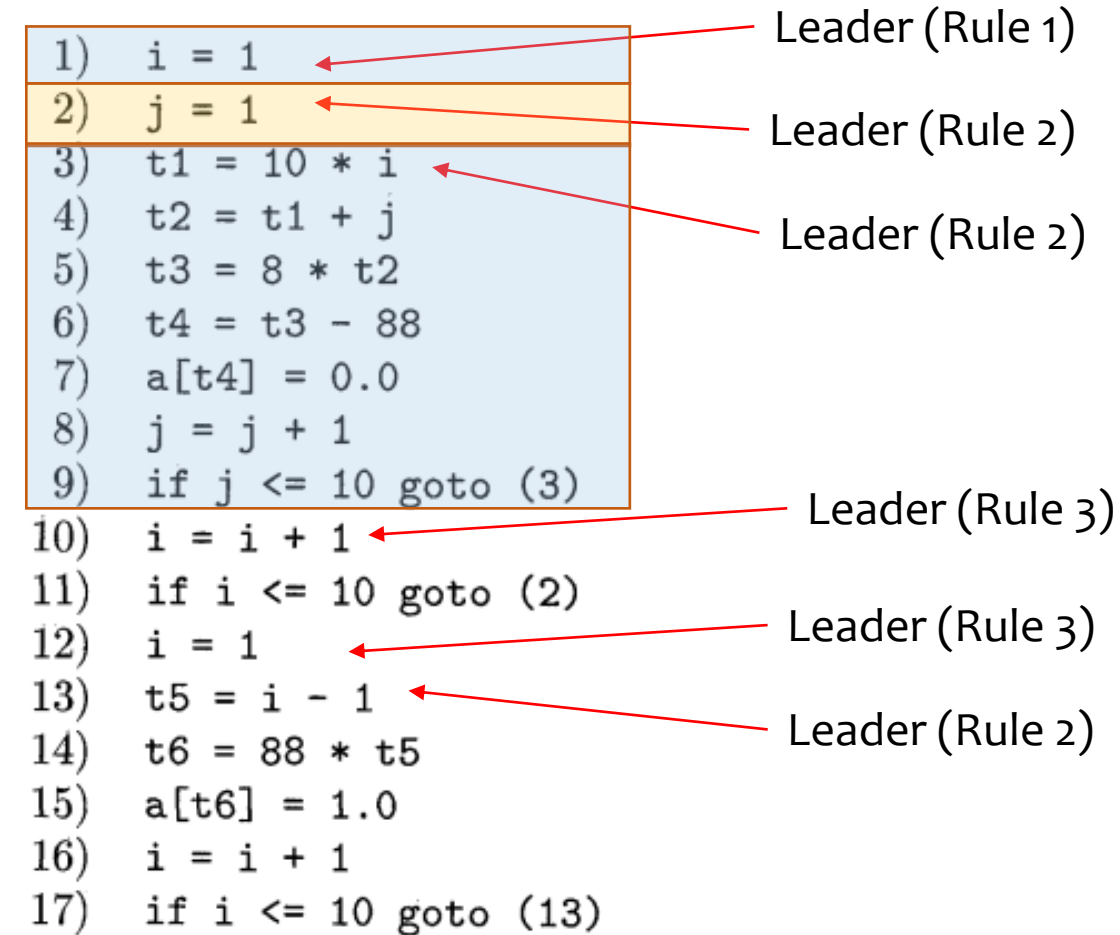
1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end



Constructing Basic Blocks

Method:

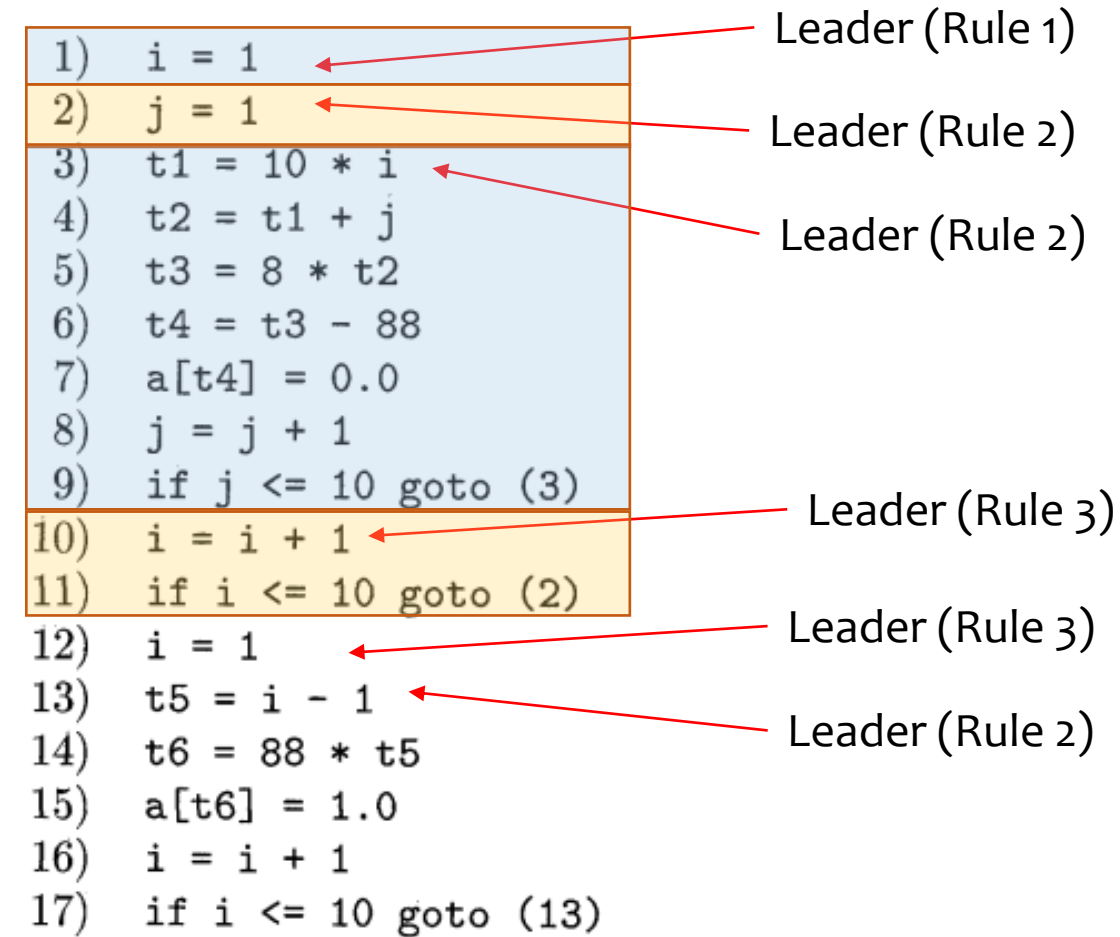
1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end



Constructing Basic Blocks

Method:

1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end



Constructing Basic Blocks

Method:

1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end

1) i = 1	← Leader (Rule 1)
2) j = 1	← Leader (Rule 2)
3) t1 = 10 * i	← Leader (Rule 2)
4) t2 = t1 + j	
5) t3 = 8 * t2	
6) t4 = t3 - 88	
7) a[t4] = 0.0	
8) j = j + 1	
9) if j <= 10 goto (3)	
10) i = i + 1	← Leader (Rule 3)
11) if i <= 10 goto (2)	
12) i = 1	← Leader (Rule 3)
13) t5 = i - 1	← Leader (Rule 2)
14) t6 = 88 * t5	
15) a[t6] = 1.0	
16) i = i + 1	
17) if i <= 10 goto (13)	

Constructing Basic Blocks

Method:

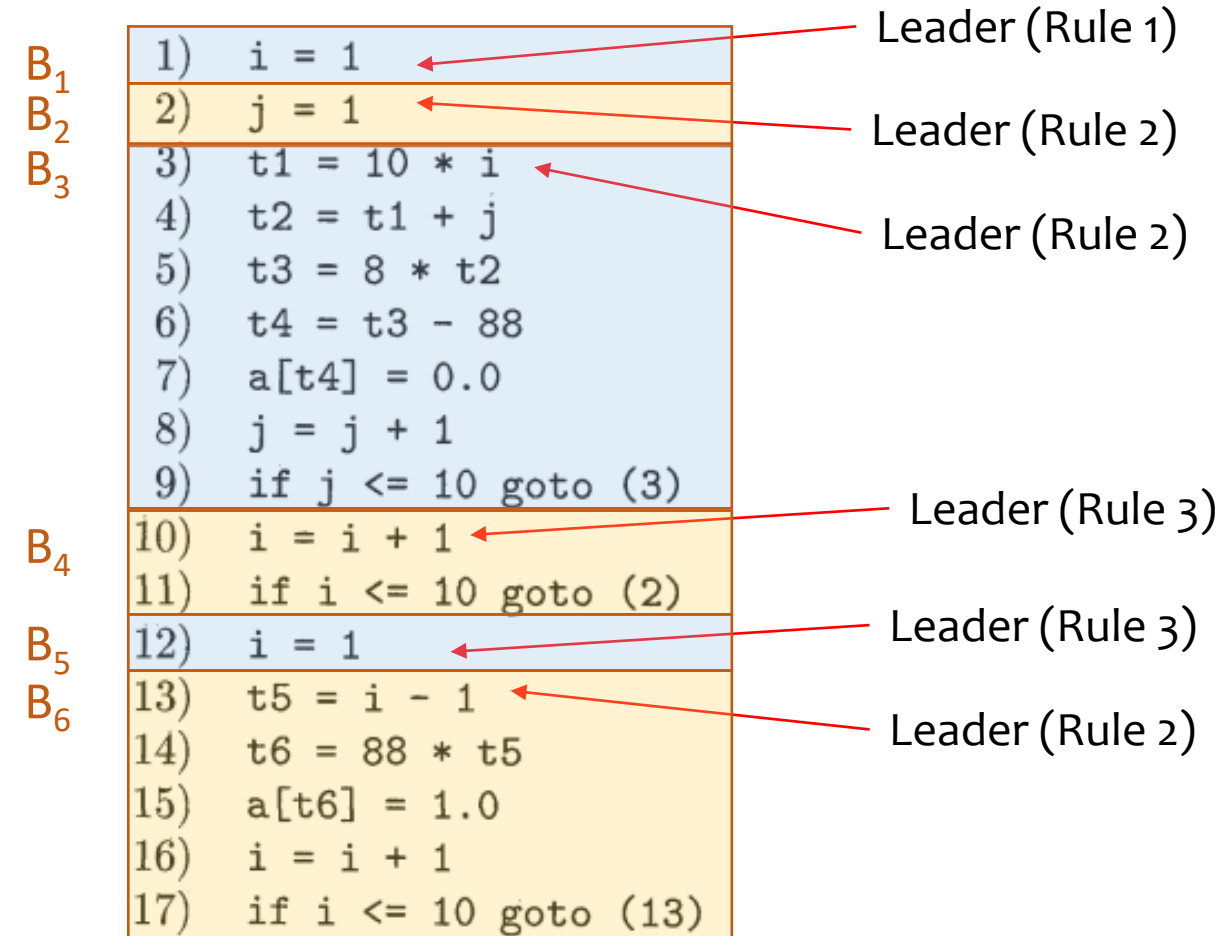
1. Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - a. The **first** instruction in a sequence is a leader
 - b. Any instruction that is the **target of a branch** is a leader
 - c. Any instruction that **follows a branch** is a leader
2. For each leader, basic block consists codes upto next leader or end

1) i = 1	Leader (Rule 1)
2) j = 1	Leader (Rule 2)
3) t1 = 10 * i	Leader (Rule 2)
4) t2 = t1 + j	
5) t3 = 8 * t2	
6) t4 = t3 - 88	
7) a[t4] = 0.0	
8) j = j + 1	
9) if j <= 10 goto (3)	
10) i = i + 1	Leader (Rule 3)
11) if i <= 10 goto (2)	
12) i = 1	Leader (Rule 3)
13) t5 = i - 1	Leader (Rule 2)
14) t6 = 88 * t5	
15) a[t6] = 1.0	
16) i = i + 1	
17) if i <= 10 goto (13)	

Constructing Basic Blocks

Method:

- Determine those instructions in the intermediate code that are the first instructions in some basic blocks – **leaders**
 - The **first** instruction in a sequence is a leader
 - Any instruction that is the **target of a branch** is a leader
 - Any instruction that **follows a branch** is a leader
- For each leader, basic block consists codes upto next leader or end



Flow Graphs

Flow Graphs

- There is an edge from a block B to block C *iff* –
 - It is possible for the first instruction in block C to immediately follow the last instruction in block B
 - There are **two ways** to construct such an edge -
 - There is a conditional or unconditional jump from the end of B to the beginning of C
 - C immediately follows B in the original order of the three – address instructions and B does not end an unconditional jump
- B is a predecessor of C and C is a successor of B

Flow Graphs

B ₁	1) i = 1
B ₂	2) j = 1
B ₃	3) t1 = 10 * i 4) t2 = t1 + j 5) t3 = 8 * t2 6) t4 = t3 - 88 7) a[t4] = 0.0 8) j = j + 1 9) if j <= 10 goto (3)
B ₄	10) i = i + 1 11) if i <= 10 goto (2)
B ₅	12) i = 1
B ₆	13) t5 = i - 1 14) t6 = 88 * t5 15) a[t6] = 1.0 16) i = i + 1 17) if i <= 10 goto (13)

Flow Graphs

B_1
 B_2
 B_3

1)	$i = 1$
2)	$j = 1$
3)	$t_1 = 10 * i$
4)	$t_2 = t_1 + j$
5)	$t_3 = 8 * t_2$
6)	$t_4 = t_3 - 88$
7)	$a[t_4] = 0.0$
8)	$j = j + 1$
9)	if $j \leq 10$ goto (3)

B_4

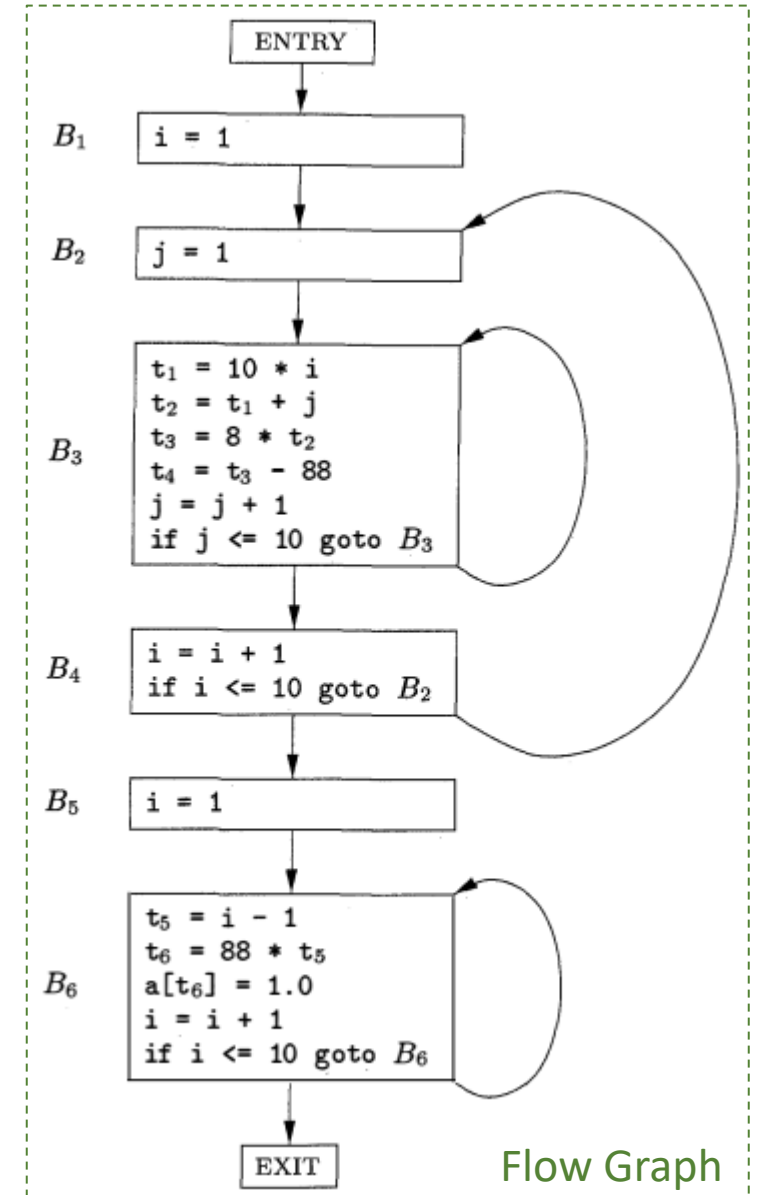
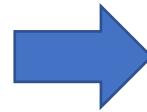
10)	$i = i + 1$
11)	if $i \leq 10$ goto (2)

B_5

12)	$i = 1$
-----	---------

B_6

13)	$t_5 = i - 1$
14)	$t_6 = 88 * t_5$
15)	$a[t_6] = 1.0$
16)	$i = i + 1$
17)	if $i \leq 10$ goto (13)



Representation of Flow Graphs

Representation of Flow Graphs

- Jumps to instruction numbers or labels are replaced by jumps to basic blocks
 - Every conditional or unconditional jump is to the leader of some basic block, it is to this block this jump will now refer
- After constructing the flow graph it is common to make substantial changes to the instructions in the various basic blocks
 - If jumps were to instructions, it would cause to fix the targets with each change
- Flow graphs can be represented by any of the data structures used for graph
- The content of nodes can be represented by
 - pointer to the leader in the array of three address instructions together with count of the number of instructions or a second pointer to the last instruction.
 - It is better to maintain a [linked list](#) as the number of instructions in a basic block are changed frequently.

Loops

Loops

- Programming-language constructs like while-statements and for-statements naturally gives rise to loops in programs.
- Virtually every program spends most of its time in executing its loops
 - It is important for a compiler to generate good code for loops
- Many code transformations depend upon the identification of “loops” in a flow graph

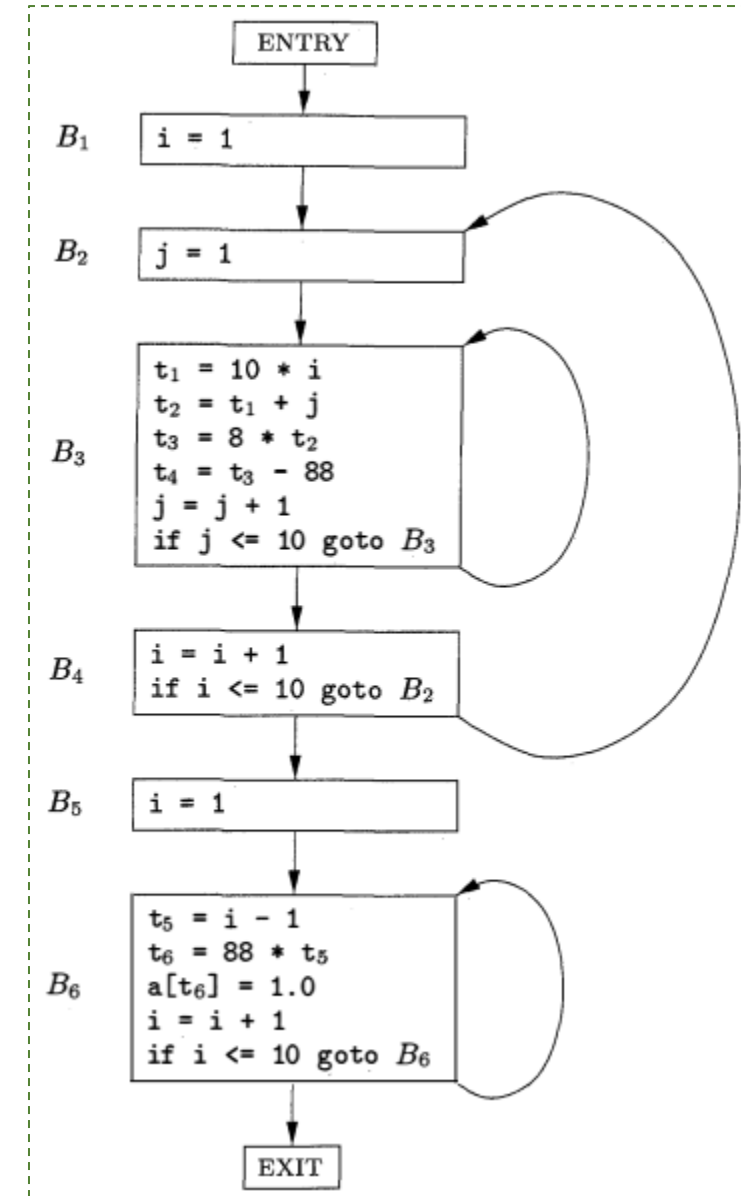
Loops

Loops

- A set of nodes L in the flow graph is a loop if L contains a node e called the **loop entry**, such that
 1. e is not **ENTRY**, the entry of the entire flow graph
 2. No node in L besides e has a predecessor outside L , *i.e.*, every path from ENTRY to any node in L goes through e
 3. Every node in L has a nonempty path, completely within L , to e
- Example: Loops in the flow graph on the right are -
 1. B_3 by itself
 2. B_6 by itself
 3. $\{B_2, B_3, B_4\}$

Loops

- A set of nodes L in the flow graph is a loop if L contains a node e called the **loop entry**, such that
 1. e is not **ENTRY**, the entry of the entire flow graph
 2. No node in L besides e has a predecessor outside L , i.e., every path from ENTRY to any node in L goes through e
 3. Every node in L has a nonempty path, completely within L , to e
- Example: Loops in the flow graph on the right are -
 1. B_3 by itself
 2. B_6 by itself
 3. $\{B_2, B_3, B_4\}$



Liveliness & Next-Use Information

Liveliness & Next-Use Information

- Knowledge of next *use* of a variable is helpful in good code generation
- If the value of a variable that is currently in a register will never be referenced subsequently, then that register can be assigned to another variable
- Definition of *use*
 - $i : x = 5$ // x is *live* at statement i
..... // no use of x
 - $j : y = x$ // statement j *uses* the value of x *computed at* i
- For each statement $x = y + z$, we want to find next use of x , y and z

Liveliness & Next-Use Information

Liveliness & Next-Use Information

- **Algorithm:** Determining liveliness & next-use information for each statement of a basic block
- **Input:** A basic block B of three-address statements. It is assumed that the symbol table initially shows all nontemporary variables in B as being live on exit
- **Output:** At each statement $i: x = y + z$, liveliness and next-use information of x, y and z will be attached with i
- **Method:** From last statement of B , scan backwards
 1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
 2. In the symbol table, set x to not live and no next use
 3. In the symbol table, set y, z to live and next use at statement i

Liveliness & Next-Use Information

From the last statement of B ,
scan backwards. At each
statement i : $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to **not live** and **no next use**
3. In the symbol table, set y, z to **live** and next use at statement i

Basic Block	Symbol Table						
	a	b	c	d	t	u	v
1. $t = a - b$							
2. $u = a - c$							
3. $v = t + u$							
4. $a = d$							
5. $d = v + u$							
	L6	L6	L6	L6	N	N	N

Liveliness & Next-Use Information

From the last statement of B ,
scan backwards. At each
statement i : $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to **not live** and **no next use**
3. In the symbol table, set y, z to **live** and next use at statement i

Basic Block	Symbol Table						
	a	b	c	d	t	u	v
1. $t = a - b$							
2. $u = a - c$							
3. $v = t + u$							
4. $a = d$							
5. $\frac{d}{L6} = \frac{v}{N} + \frac{u}{N}$							
	L6	L6	L6	L6	N	N	N

Liveliness & Next-Use Information

From the last statement of B ,
scan backwards. At each
statement i : $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to **not live** and **no next use**
3. In the symbol table, set y, z to **live** and next use at statement i

Basic Block	Symbol Table						
	a	b	c	d	t	u	v
1. $t = a - b$							
2. $u = a - c$							
3. $v = t + u$							
4. $a = d$							
5. $\frac{d}{L6} = \frac{v}{N} + \frac{u}{N}$	L6	L6	L6	N	N	L5	L5
	L6	L6	L6	L6	N	N	N

Liveliness & Next-Use Information

From the last statement of B ,
scan backwards. At each
statement i : $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to **not live** and **no next use**
3. In the symbol table, set y, z to **live** and next use at statement i

Basic Block	Symbol Table						
	a	b	c	d	t	u	v
1. $t = a - b$							
2. $u = a - c$							
3. $v = t + u$							
4. $\frac{a}{L6} = \frac{d}{N}$							
5. $\frac{d}{L6} = \frac{v}{N} + \frac{u}{N}$							

Liveliness & Next-Use Information

From the last statement of B ,
scan backwards. At each
statement i : $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to **not live** and **no next use**
3. In the symbol table, set y, z to **live** and next use at statement i

Basic Block	Symbol Table						
	a	b	c	d	t	u	v
1. $t = a - b$							
2. $u = a - c$							
3. $v = t + u$							
4. $\frac{a}{L6} = \frac{d}{N}$	N	L6	L6	L4	N	L5	L5
5. $\frac{d}{L6} = \frac{v}{N} + \frac{u}{N}$	L6	L6	L6	N	N	L5	L5
	L6	L6	L6	L6	N	N	N

Liveliness & Next-Use Information

From the last statement of B ,
scan backwards. At each
statement i : $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to **not live** and **no next use**
3. In the symbol table, set y, z to **live** and next use at statement i

Basic Block	Symbol Table						
	a	b	c	d	t	u	v
1. $t = a - b$							
2. $u = a - c$							
3. $\frac{v}{L5} = \frac{t}{N} + \frac{u}{L5}$	N	L6	L6	L4	N	L5	L5
4. $\frac{a}{L6} = \frac{d}{N}$	L6	L6	L6	N	N	L5	L5
5. $\frac{d}{L6} = \frac{v}{N} + \frac{u}{N}$	L6	L6	L6	L6	N	N	N

Liveliness & Next-Use Information

From the last statement of B ,
scan backwards. At each
statement i : $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to **not live** and **no next use**
3. In the symbol table, set y, z to **live** and next use at statement i

Basic Block	Symbol Table						
	a	b	c	d	t	u	v
1. $t = a - b$							
2. $u = a - c$							
3. $\frac{v}{\text{L5}} = \frac{t}{\text{N}} + \frac{u}{\text{L5}}$	N	L6	L6	L4	L3	L3	N
4. $\frac{a}{\text{L6}} = \frac{d}{\text{N}}$	N	L6	L6	L4	N	L5	L5
5. $\frac{d}{\text{L6}} = \frac{v}{\text{N}} + \frac{u}{\text{N}}$	L6	L6	L6	N	N	L5	L5
	L6	L6	L6	L6	N	N	N

Liveliness & Next-Use Information

From the last statement of B ,
scan backwards. At each
statement i : $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to **not live** and **no next use**
3. In the symbol table, set y, z to **live** and next use at statement i

Basic Block	Symbol Table						
	a	b	c	d	t	u	v
1. $t = a - b$							
2. $\frac{u}{L3} = \frac{a}{N} - \frac{c}{L6}$	N	L6	L6	L4	L3	L3	N
3. $\frac{v}{L5} = \frac{t}{N} + \frac{u}{L5}$	N	L6	L6	L4	N	L5	L5
4. $\frac{a}{L6} = \frac{d}{N}$	L6	L6	L6	N	N	L5	L5
5. $\frac{d}{L6} = \frac{v}{N} + \frac{u}{N}$	L6	L6	L6	L6	N	N	N

Liveliness & Next-Use Information

From the last statement of B ,
scan backwards. At each
statement i : $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to **not live** and **no next use**
3. In the symbol table, set y, z to **live** and next use at statement i

Basic Block	Symbol Table						
	a	b	c	d	t	u	v
1. $t = a - b$							
	L2	L6	L2	L4	L3	N	N
2. $\frac{u}{L3} = \frac{a}{N} - \frac{c}{L6}$	N	L6	L6	L4	L3	L3	N
3. $\frac{v}{L5} = \frac{t}{N} + \frac{u}{L5}$	N	L6	L6	L4	N	L5	L5
4. $\frac{a}{L6} = \frac{d}{N}$	L6	L6	L6	N	N	L5	L5
5. $\frac{d}{L6} = \frac{v}{N} + \frac{u}{N}$	L6	L6	L6	L6	N	N	N

Liveliness & Next-Use Information

From the last statement of B ,
scan backwards. At each
statement i : $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to **not live** and **no next use**
3. In the symbol table, set y, z to **live** and next use at statement i

Basic Block	Symbol Table						
	a	b	c	d	t	u	v
1. $\frac{t}{L3} = \frac{a}{L2} - \frac{b}{L6}$	L2	L6	L2	L4	L3	N	N
2. $\frac{u}{L3} = \frac{a}{N} - \frac{c}{L6}$	N	L6	L6	L4	L3	L3	N
3. $\frac{v}{L5} = \frac{t}{N} + \frac{u}{L5}$	N	L6	L6	L4	N	L5	L5
4. $\frac{a}{L6} = \frac{d}{N}$	L6	L6	L6	N	N	L5	L5
5. $\frac{d}{L6} = \frac{v}{N} + \frac{u}{N}$	L6	L6	L6	L6	N	N	N

Liveliness & Next-Use Information

From the last statement of B ,
 scan backwards. At each
 statement i : $x=y+z$ -

1. Attach current liveliness & Next-use of x, y and z found in symbol table to statement i
2. In the symbol table, set x to **not live** and **no next use**
3. In the symbol table, set y, z to **live** and next use at statement i

Basic Block	Symbol Table						
	a	b	c	d	t	u	v
1. $t = a - b$ $\overline{L3} \quad \overline{L2} \quad \overline{L6}$	L1	L1	L2	L4	N	N	N
2. $u = a - c$ $\overline{L3} \quad \overline{N} \quad \overline{L6}$	L2	L6	L2	L4	L3	N	N
3. $v = t + u$ $\overline{L5} \quad \overline{N} \quad \overline{L5}$	N	L6	L6	L4	L3	L3	N
4. $a = d$ $\overline{L6} \quad \overline{N}$	N	L6	L6	L4	N	L5	L5
5. $d = v + u$ $\overline{L6} \quad \overline{N} \quad \overline{N}$	L6	L6	L6	N	N	L5	L5
	L6	L6	L6	L6	N	N	N

Optimization of Basic Blocks

Optimization of Basic Blocks

- Optimization within each basic block is also called **local optimization**
- Some code improving transformations are -
 - **Common Subexpression Elimination**
 - **Dead Code Elimination**
 - **Reordering of statements to reduce the time of a temporary variable in a register**
 - **Simplify Algebraic Computation**

DAG representation of Basic Blocks

DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block

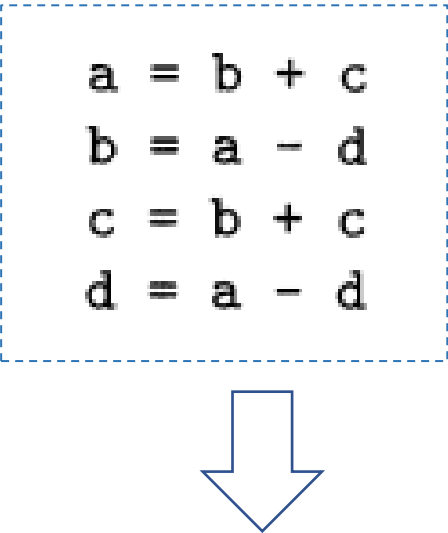
DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block

```
a = b + c
b = a - d
c = b + c
d = a - d
```

DAG representation of Basic Blocks

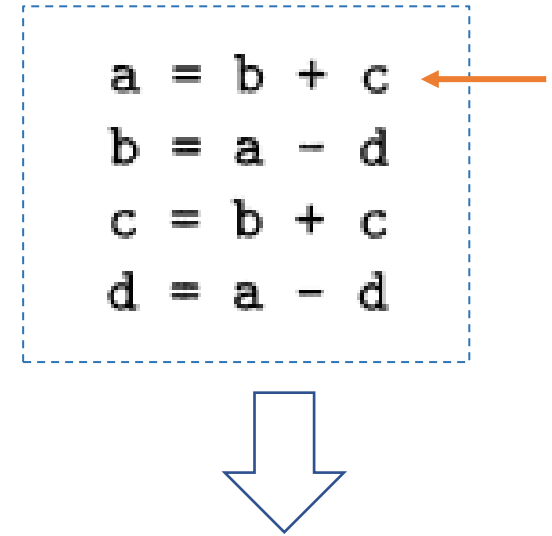
- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block



```
a = b + c
b = a - d
c = b + c
d = a - d
```

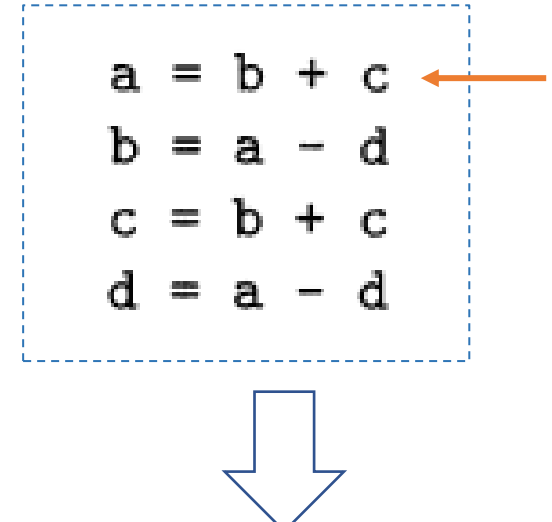
DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block



DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block



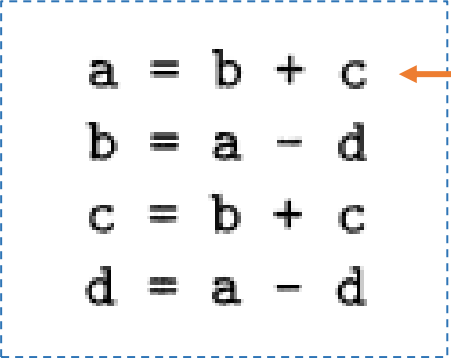
```
a = b + c
b = a - d
c = b + c
d = a - d
```

b_0

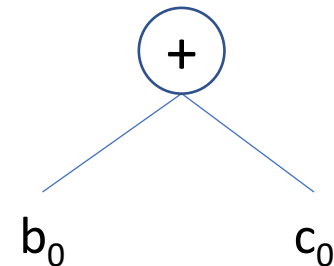
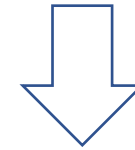
c_0

DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block

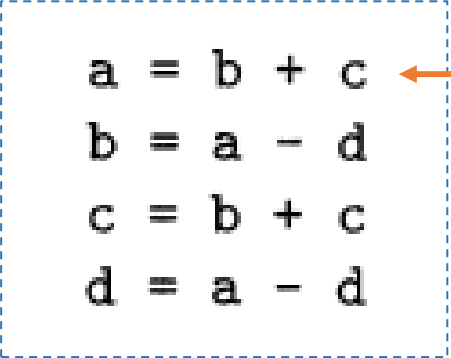


```
a = b + c
b = a - d
c = b + c
d = a - d
```

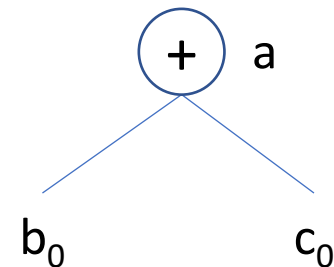
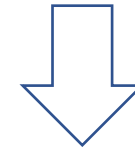


DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block



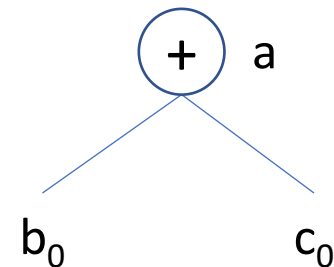
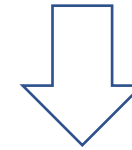
```
a = b + c
b = a - d
c = b + c
d = a - d
```



DAG representation of Basic Blocks

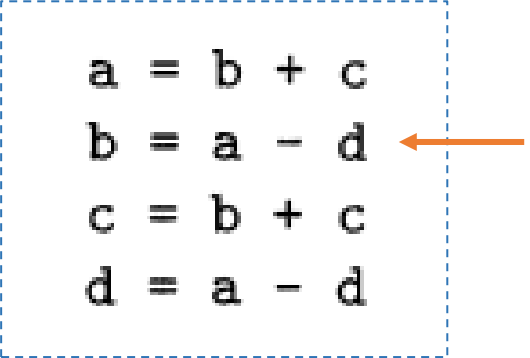
- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block

$a = b + c$
 $b = a - d$
 $c = b + c$
 $d = a - d$

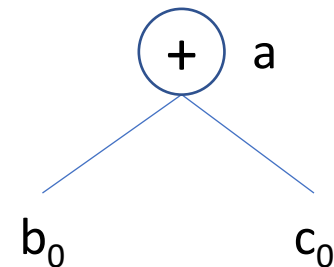
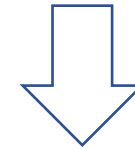


DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block

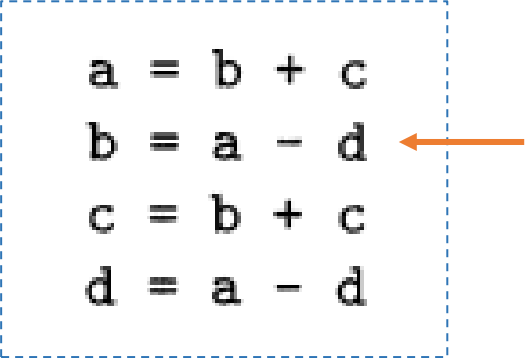


```
a = b + c
b = a - d
c = b + c
d = a - d
```

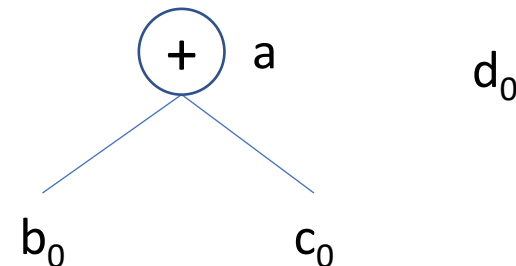
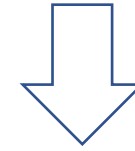


DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block



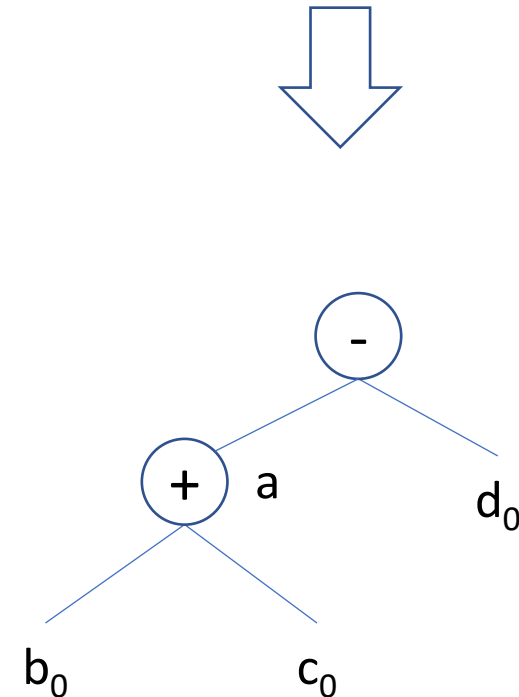
```
a = b + c
b = a - d
c = b + c
d = a - d
```



DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block

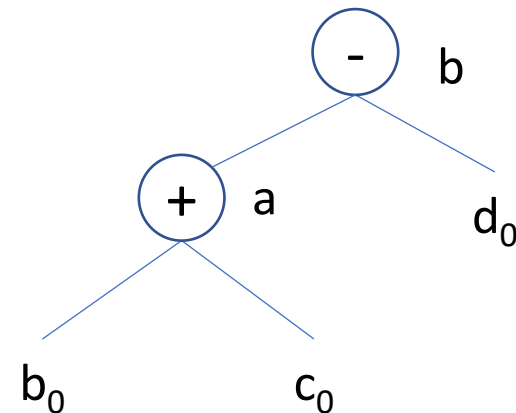
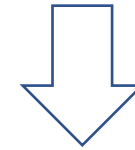
```
a = b + c
b = a - d ←
c = b + c
d = a - d
```



DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block

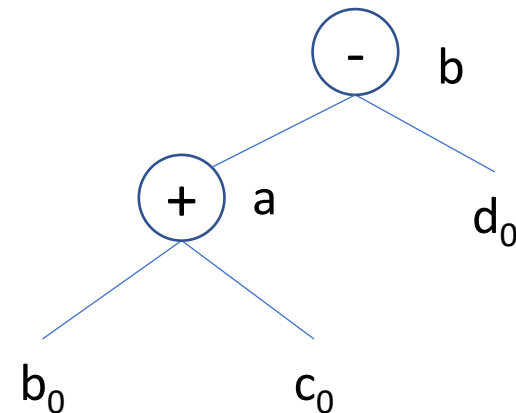
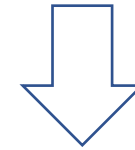
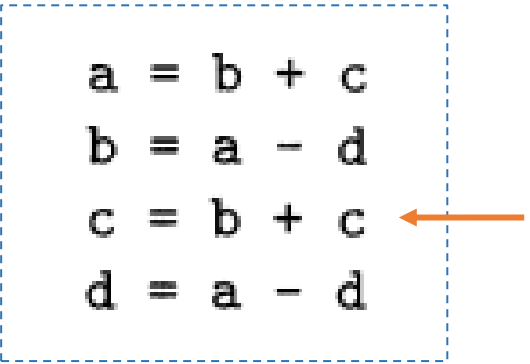
```
a = b + c
b = a - d
c = b + c
d = a - d
```



DAG representation of Basic Blocks

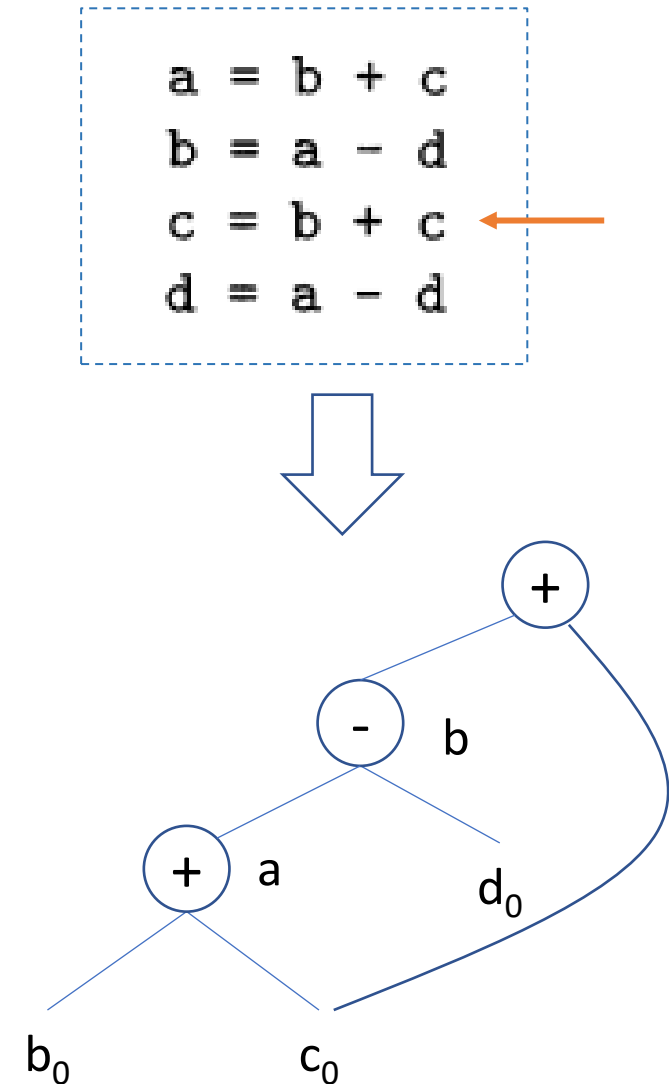
- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block

```
a = b + c
b = a - d
c = b + c
d = a - d
```



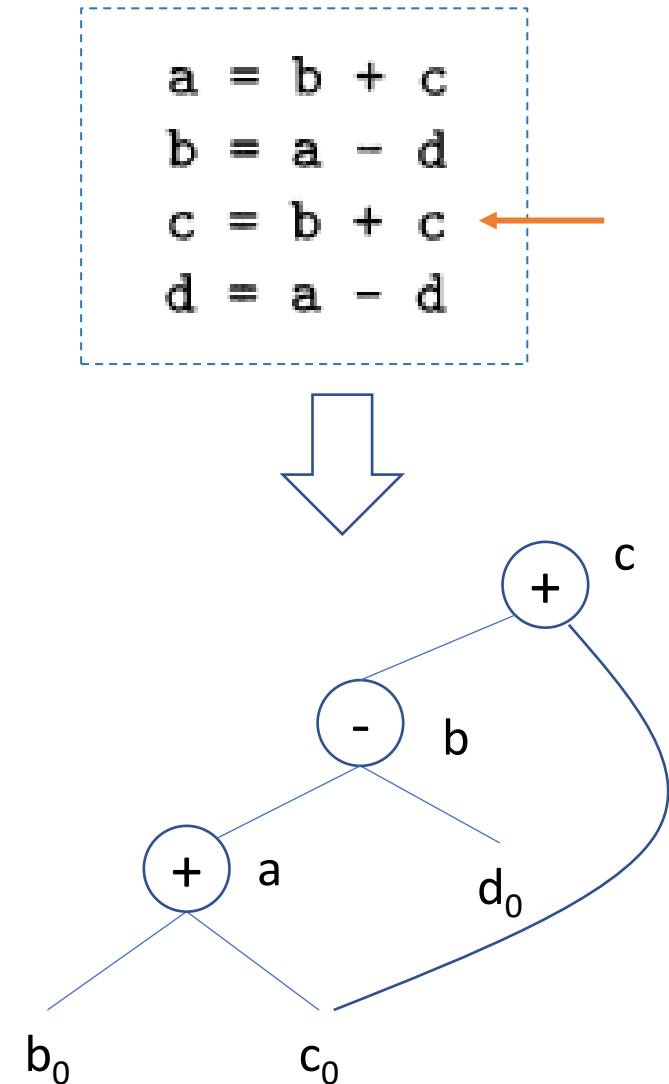
DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block



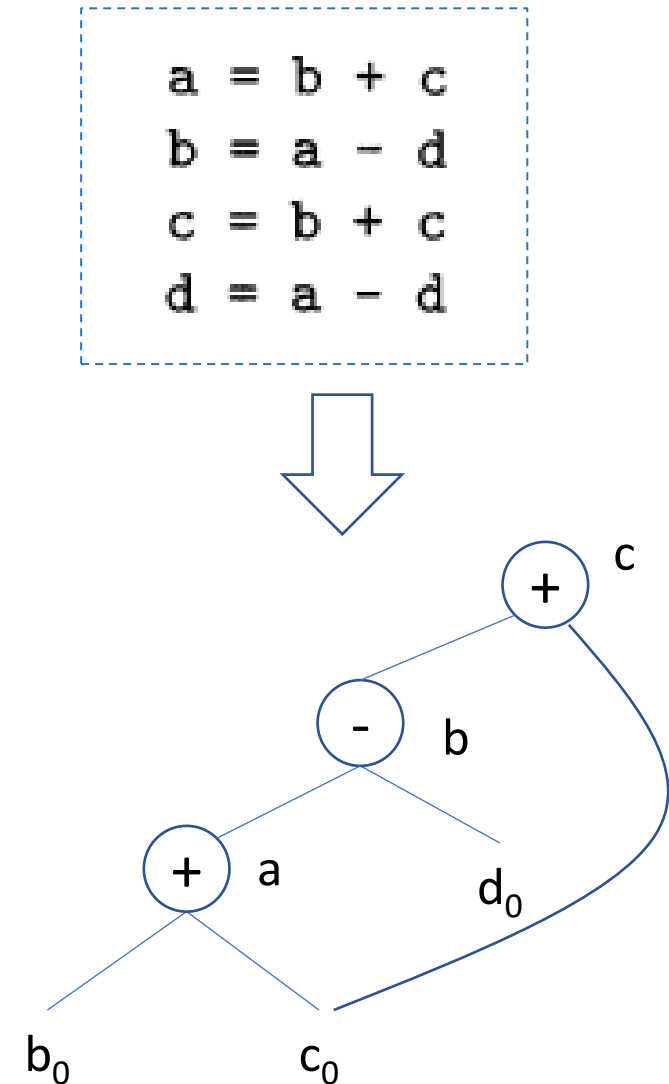
DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block



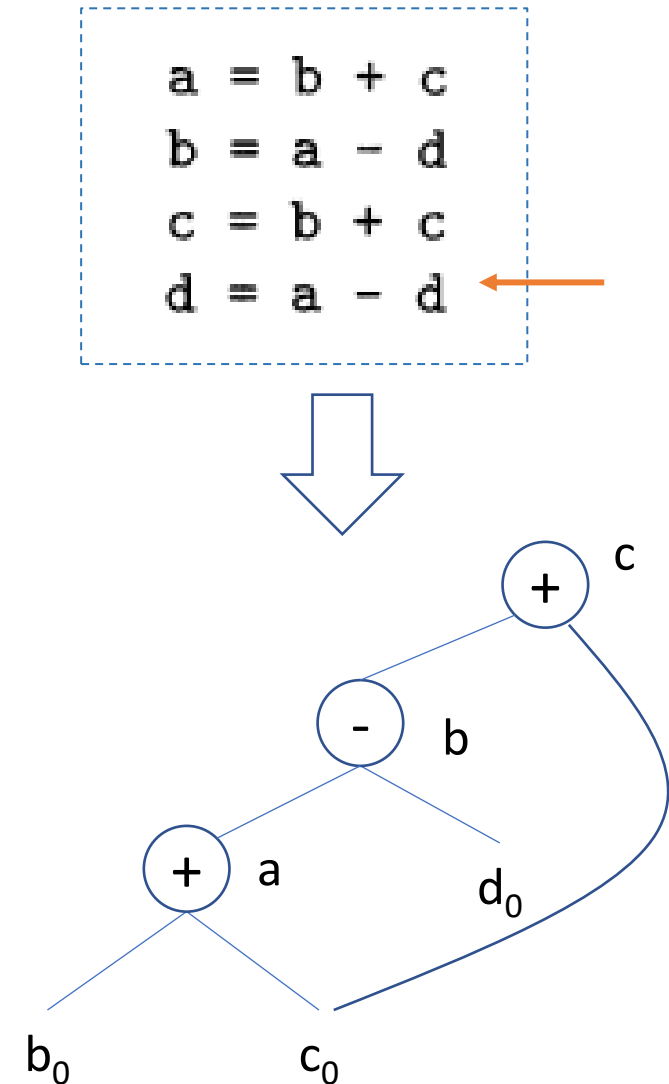
DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block



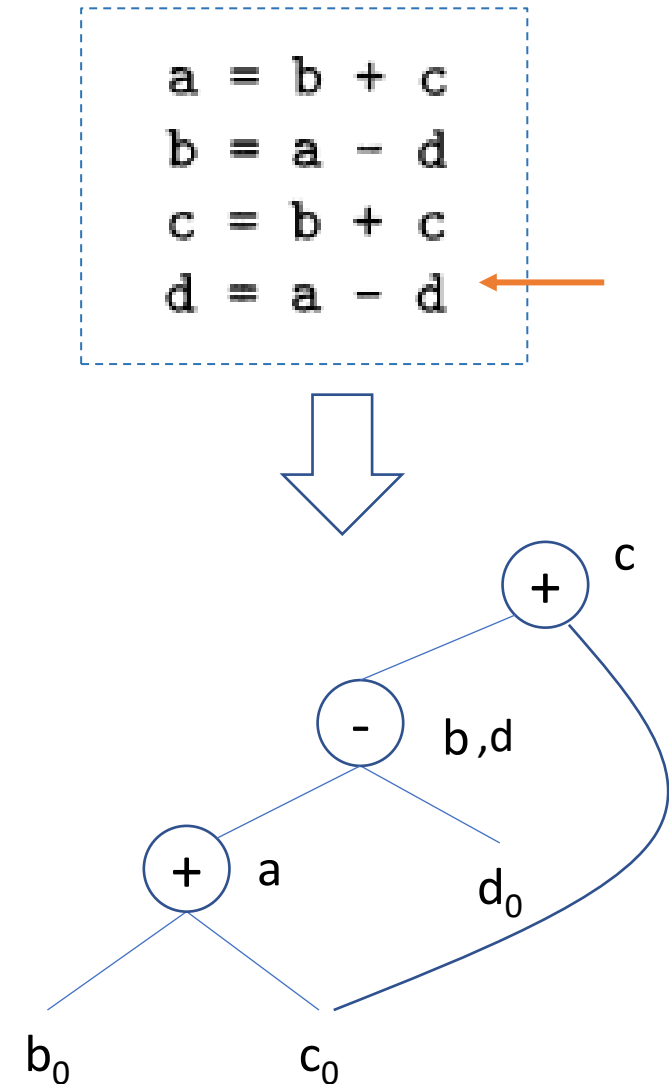
DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block



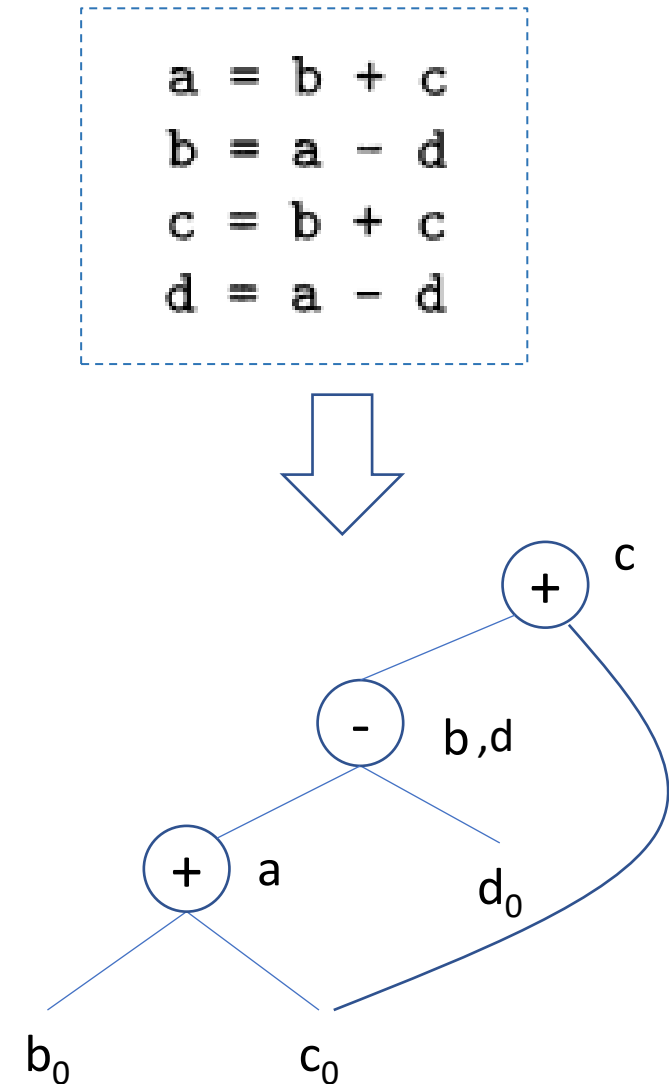
DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block

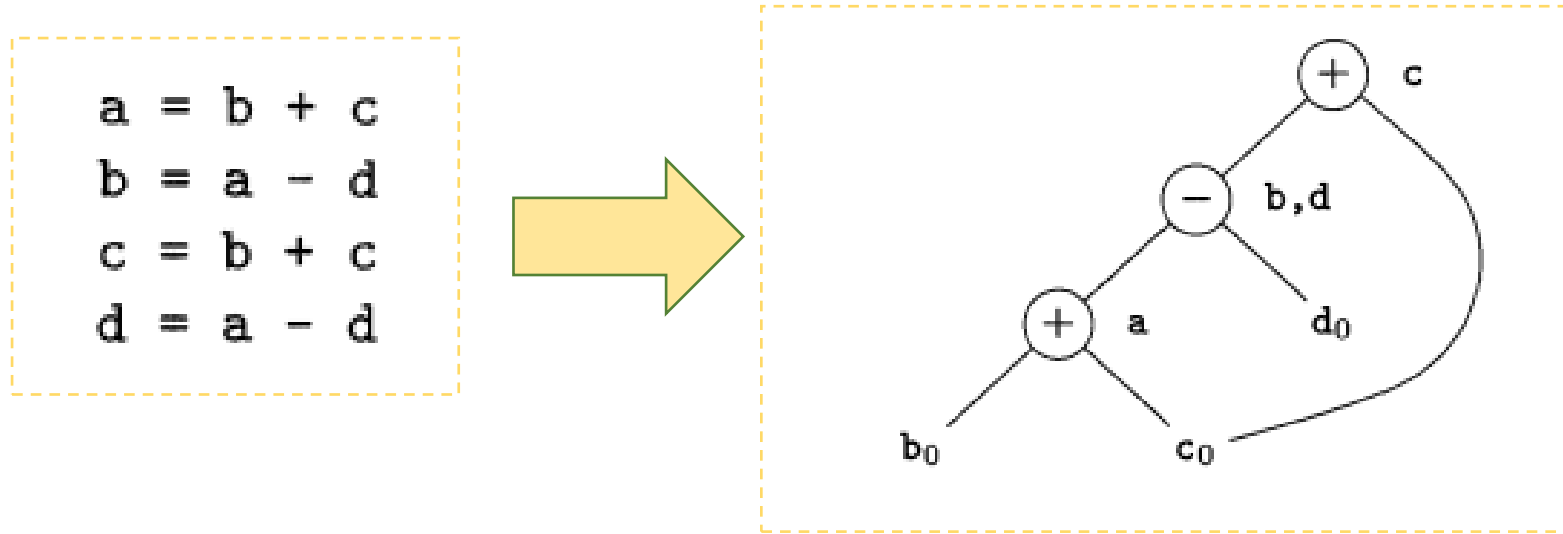


DAG representation of Basic Blocks

- Many important techniques for local optimization begin by transforming a basic block into a DAG
- We can construct DAG for a basic block as follows
 - There is a node in the DAG for each of the initial values of the variables appearing in the basic block
 - There is a node N associated with each statement s within the block. The children of N are those nodes corresponding to statements that are the last definitions, prior to s , of the operands used by s
 - Node N is labeled by the operator applied at s , and also attached to N is the list of variables for which it is the last definition within the block
 - Certain nodes are designated *output nodes*. These are the nodes whose variables that are *live on exit* from the block

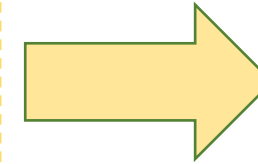
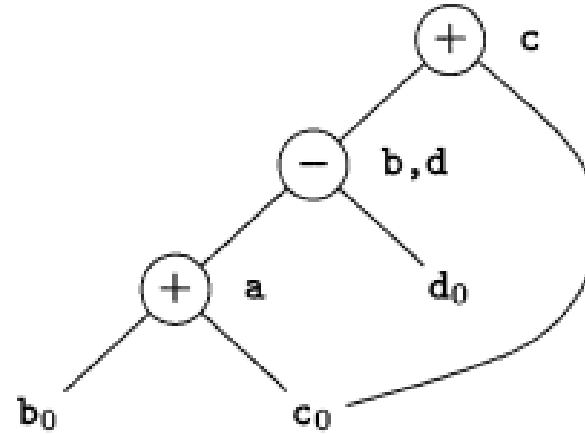
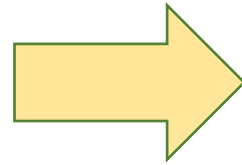


Eliminate Local Common Subexpressions



Eliminate Local Common Subexpressions

$a = b + c$
 $b = a - d$
 $c = b + c$
 $d = a - d$

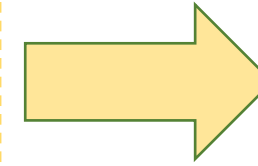
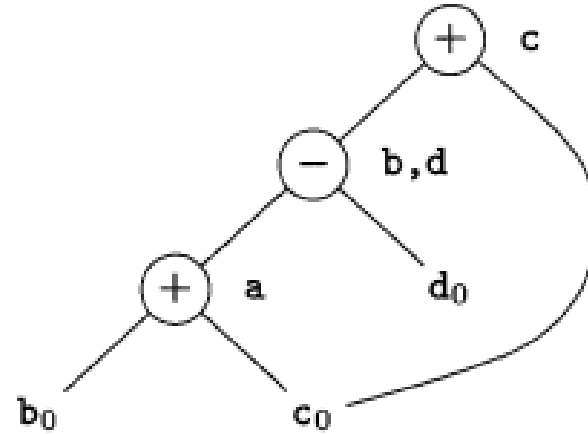
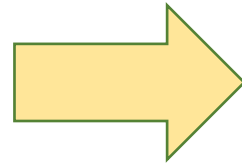


*b is not live
on exit*

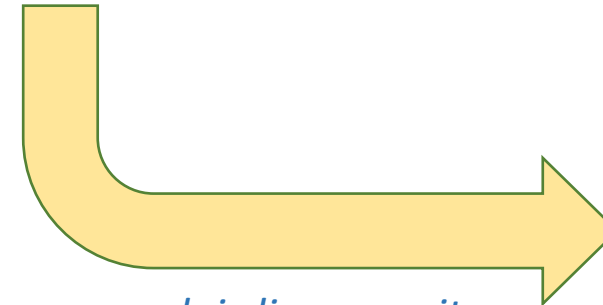
$a = b + c$
 $d = a - d$
 $c = d + c$

Eliminate Local Common Subexpressions

$a = b + c$
 $b = a - d$
 $c = b + c$
 $d = a - d$



$a = b + c$
 $d = a - d$
 $c = d + c$



b is live on exit

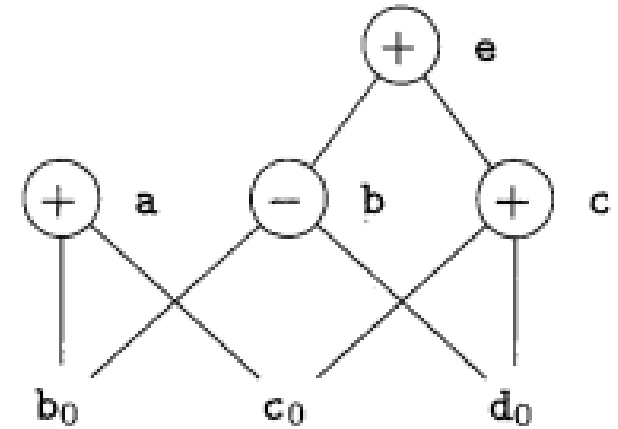
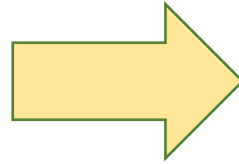
$a = b + c$
 $d = a - d$
 $b = d$
 $c = d + c$

Finding Local Common Subexpressions

```
a = b + c;  
b = b - d  
c = c + d  
e = b + c
```

Finding Local Common Subexpressions

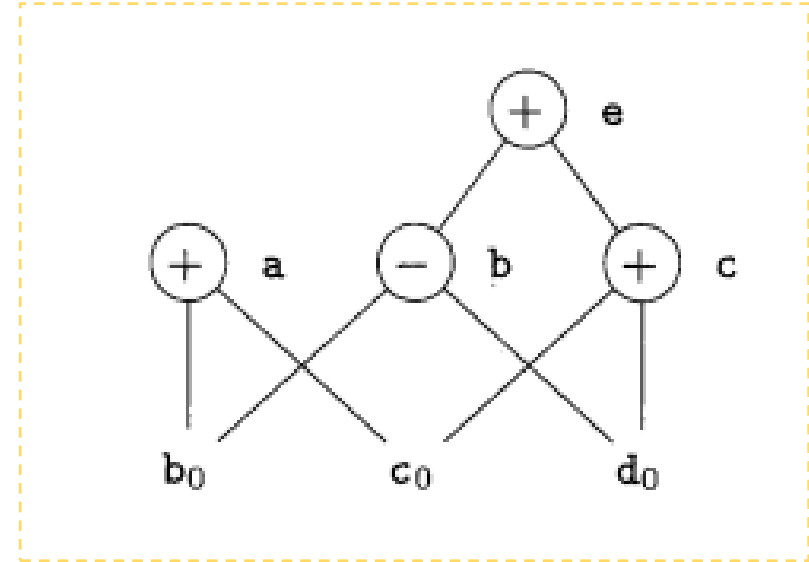
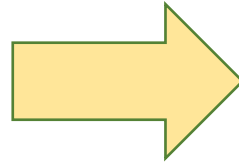
`a = b + c;`
`b = b - d`
`c = c + d`
`e = b + c`



Finding Local Common Subexpressions

- Another Example:

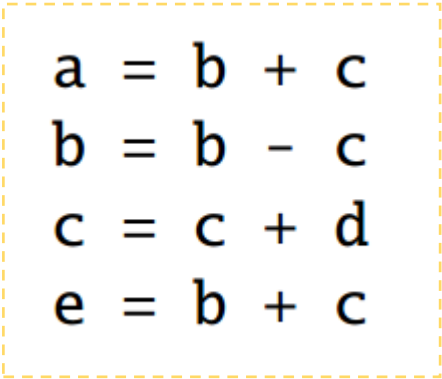
```
a = b + c;  
b = b - d;  
c = c + d;  
e = b + c
```



- What will happen in this case?
 - DAG method will fail to recognize detect common subexpression of 1st and 4th instruction

Dead Code Elimination

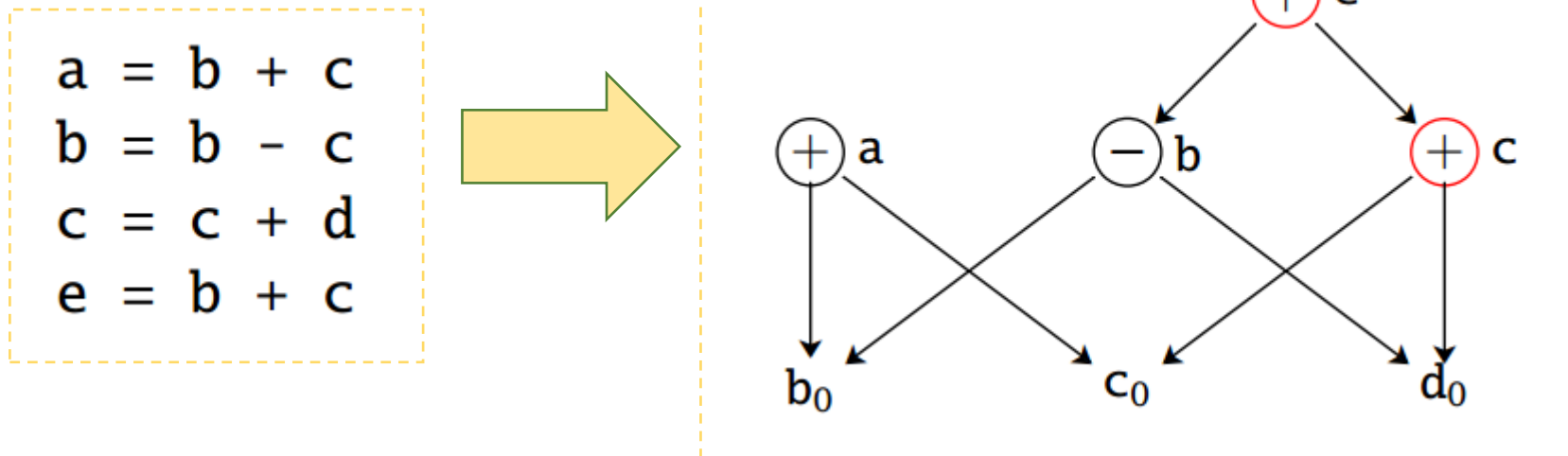
- Eliminate instructions that compute values that are never used
- Delete the nodes with no ancestor and no attached live variables from the DAG



```
a = b + c  
b = b - c  
c = c + d  
e = b + c
```

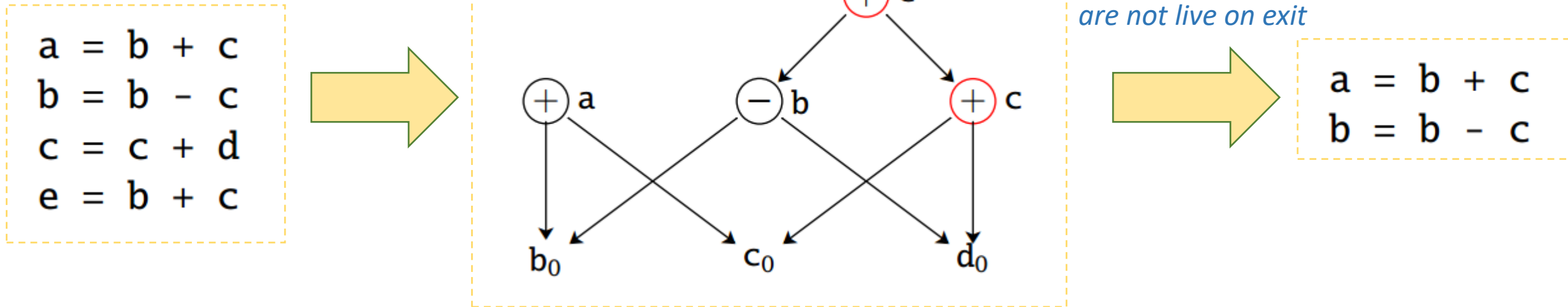
Dead Code Elimination

- Eliminate instructions that compute values that are never used
- Delete the nodes with no ancestor and no attached live variables from the DAG



Dead Code Elimination

- Eliminate instructions that compute values that are never used
- Delete the nodes with no ancestor and no attached live variables from the DAG



Use of Algebraic Identities

Use of Algebraic Identities

- Apply algebraic identities to eliminate computation
- Local reduction in strength
- Constant Folding by evaluating constant expressions at compile time
 - Example: replace $2 * 3.14$ by 6.28

Use of Algebraic Identities

- Apply **algebraic identities** to eliminate computation

$$x + 0 = 0 + x = x$$

$$x \times 1 = 1 \times x = x$$

$$x - 0 = x$$

$$x/1 = x$$

- **Local reduction in strength**

- **Constant Folding** by evaluating constant expressions at compile time
 - **Example:** replace 2×3.14 by 6.28

Use of Algebraic Identities

- Apply **algebraic identities** to eliminate computation

$$x + 0 = 0 + x = x$$

$$x \times 1 = 1 \times x = x$$

$$x - 0 = x$$

$$x/1 = x$$

- **Local reduction in strength**

EXPENSIVE

CHEAPER

$$x^2$$

=

$$x \times x$$

$$2 \times x$$

=

$$x + x$$

$$x/2$$

=

$$x \times 0.5$$

- **Constant Folding** by evaluating constant expressions at compile time
 - **Example:** replace 2×3.14 by 6.28

Use of Algebraic Identities

Use of Algebraic Identities

- **Commutativity** with local common subexpression
- **Associativity** with composite expression

Use of Algebraic Identities

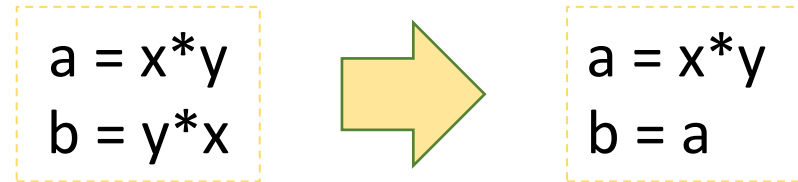
- **Commutativity** with local common subexpression

$$\begin{array}{l} a = x * y \\ b = y * x \end{array}$$

- **Associativity** with composite expression

Use of Algebraic Identities

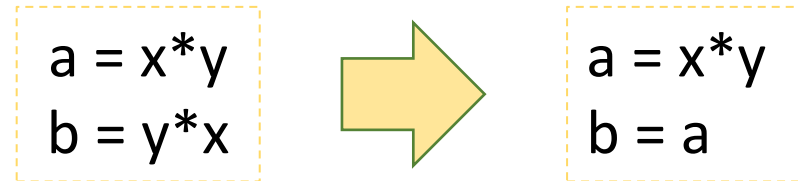
- **Commutativity** with local common subexpression



- **Associativity** with composite expression

Use of Algebraic Identities

- **Commutativity** with local common subexpression

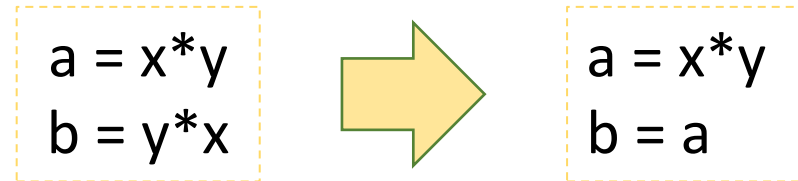


- **Associativity** with composite expression

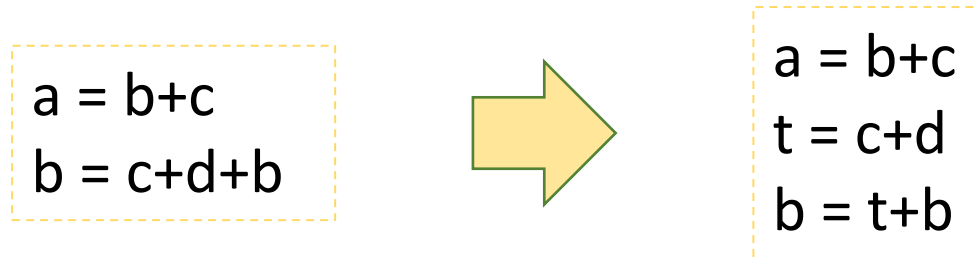
The diagram shows a dashed box containing the equations $a = b + c$ and $b = c + d + b$. This represents a composite expression where a sub-expression is used multiple times.

Use of Algebraic Identities

- **Commutativity** with local common subexpression

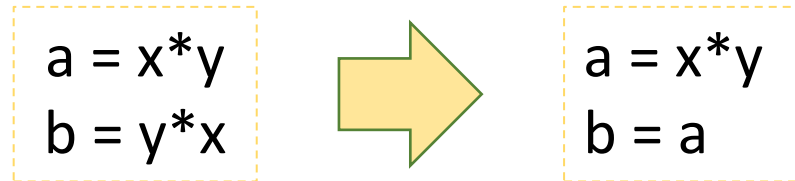


- **Associativity** with composite expression

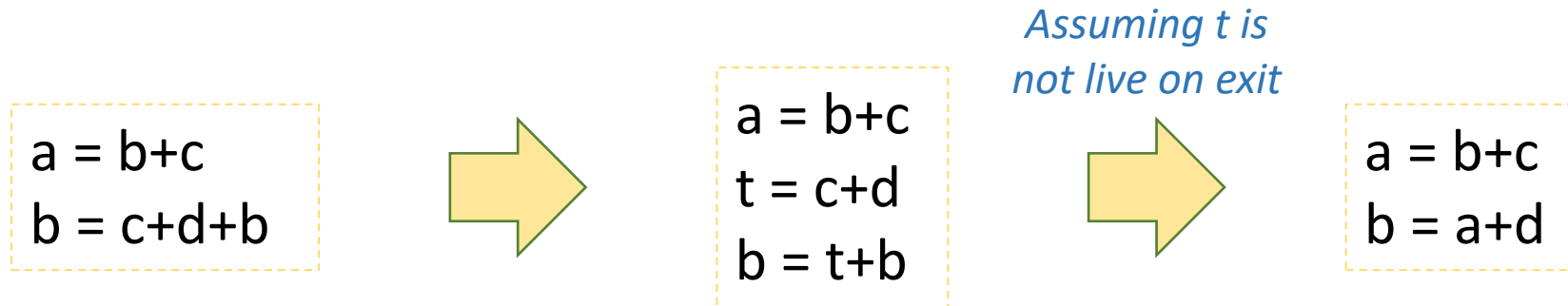


Use of Algebraic Identities

- **Commutativity** with local common subexpression



- **Associativity** with composite expression



Representation of Array References

1. An assignment from an array, like $x = a[i]$, is represented by creating a node with operator $=[]$ and two children representing the initial value of the array, a_0 in this case, and the index i . Variable x becomes a label of this new node.
2. An assignment to an array, like $a[j] = y$, is represented by a new node with operator $[]=$ and three children representing a_0 , j and y . There is no variable labeling this node. What is different is that the creation of this node *kills* all currently constructed nodes whose value depends on a_0 . A node that has been killed cannot receive any more labels; that is, it cannot become a common subexpression.

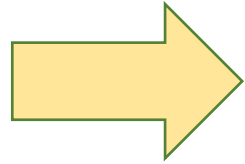
Representation of Array References

Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```

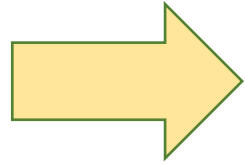
Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```



Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```

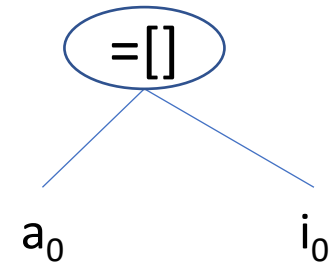
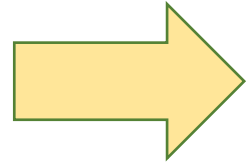


a_0

i_0

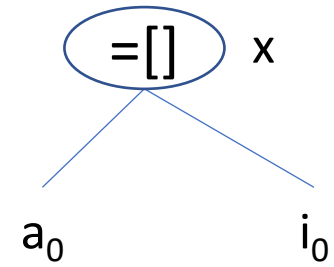
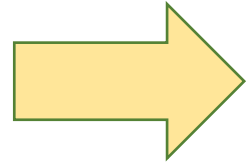
Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```



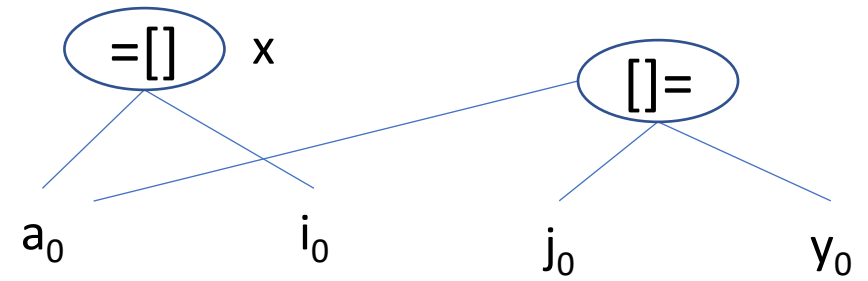
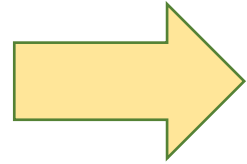
Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```



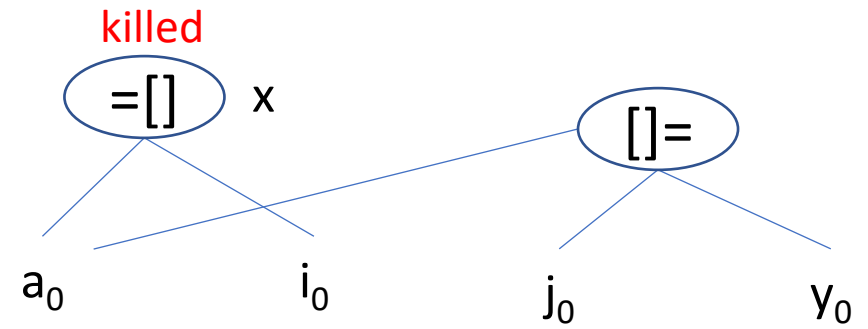
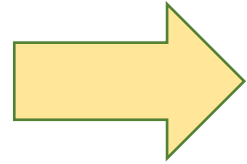
Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```



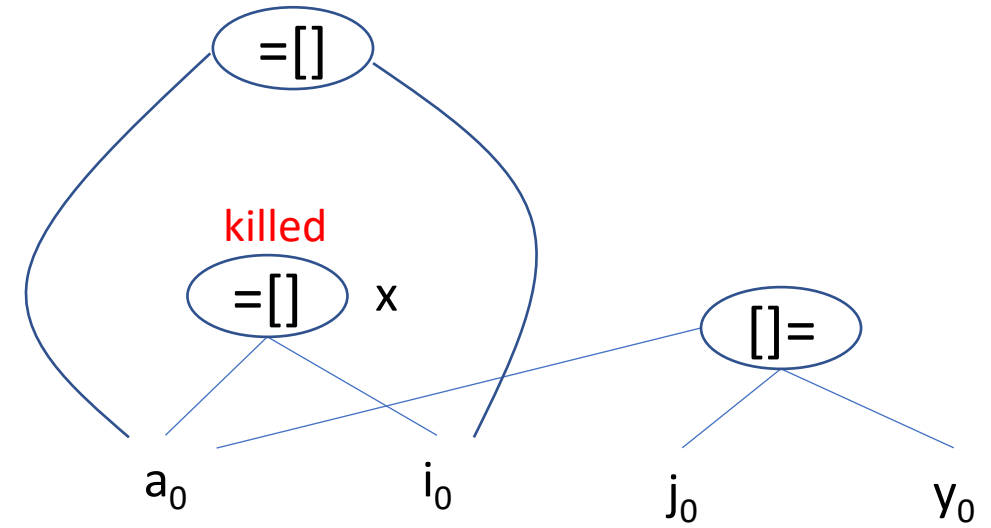
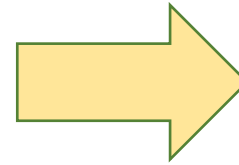
Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```



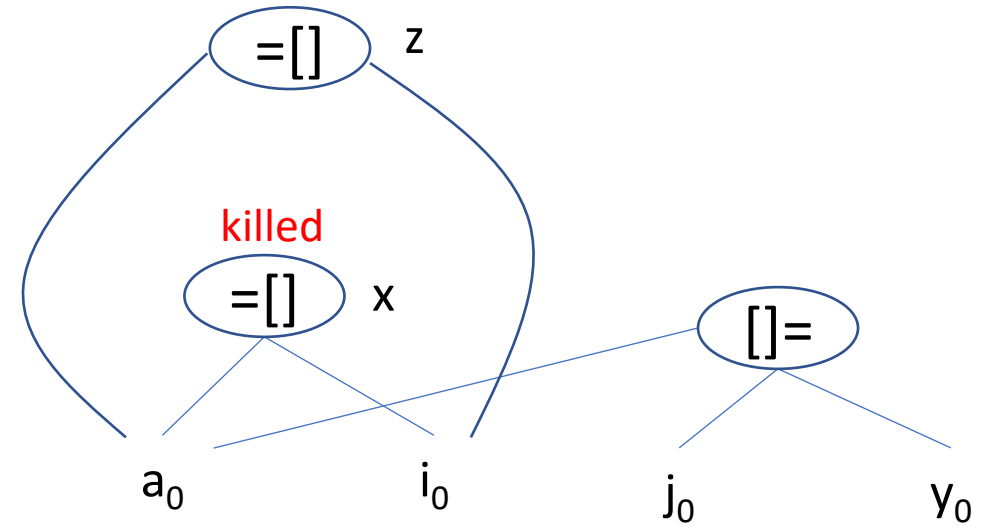
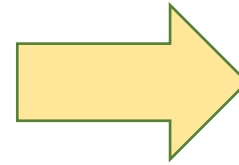
Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```



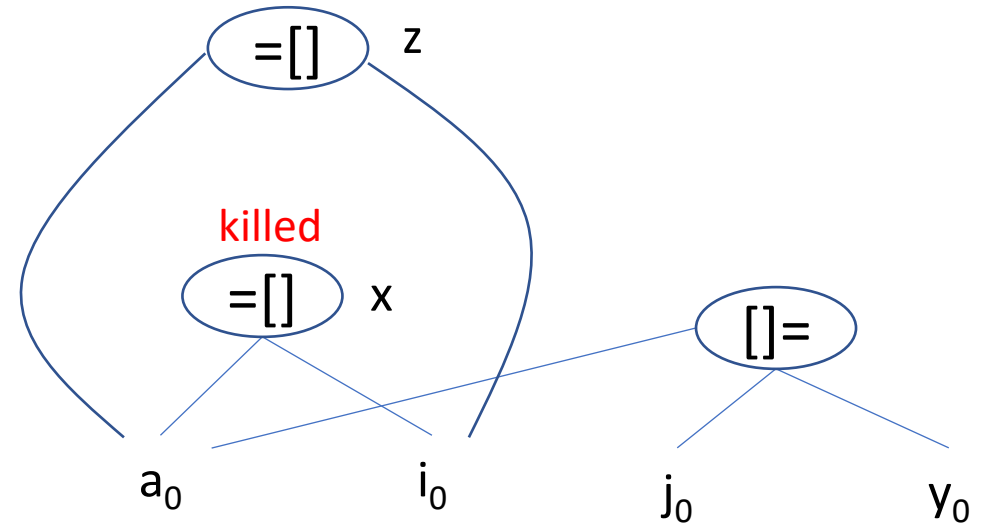
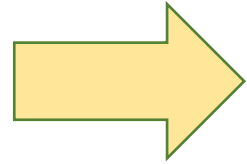
Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```

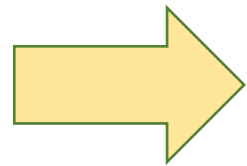


Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```

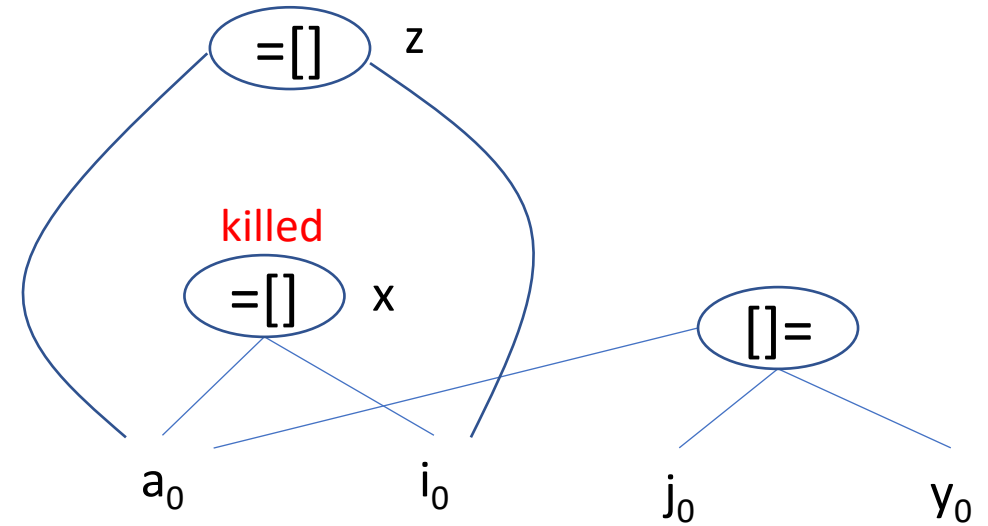
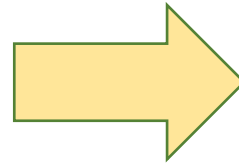


```
b = 12 + a  
x = b[i]  
b[j] = y
```

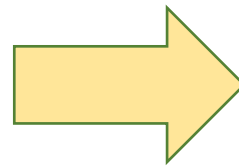


Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```



```
b = 12 + a  
x = b[i]  
b[j] = y
```

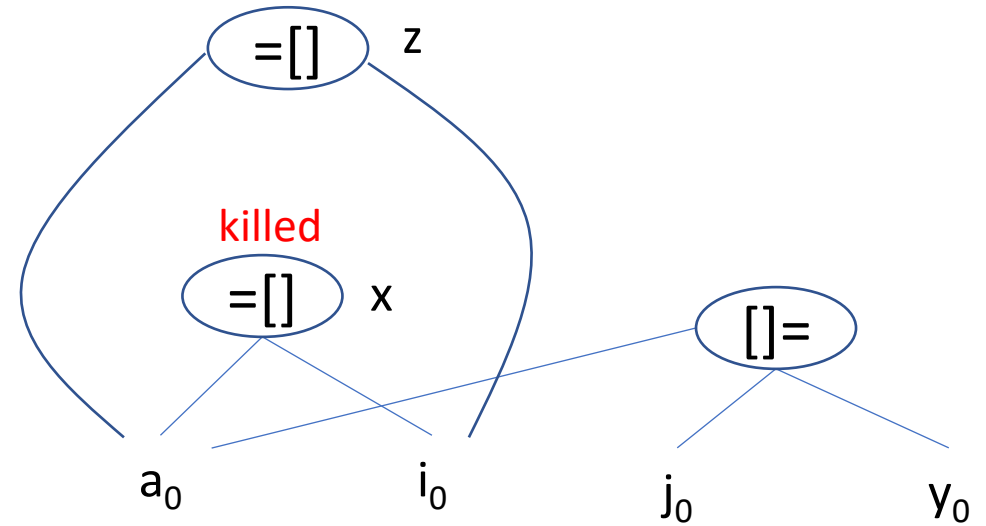
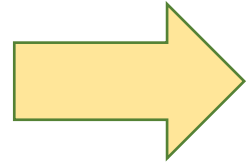


12

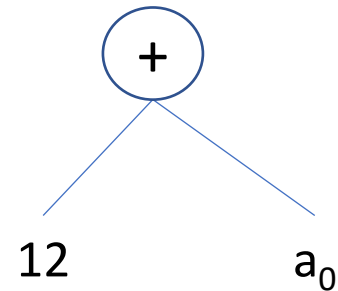
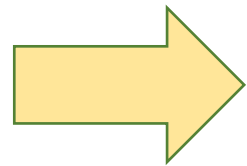
a_0

Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```

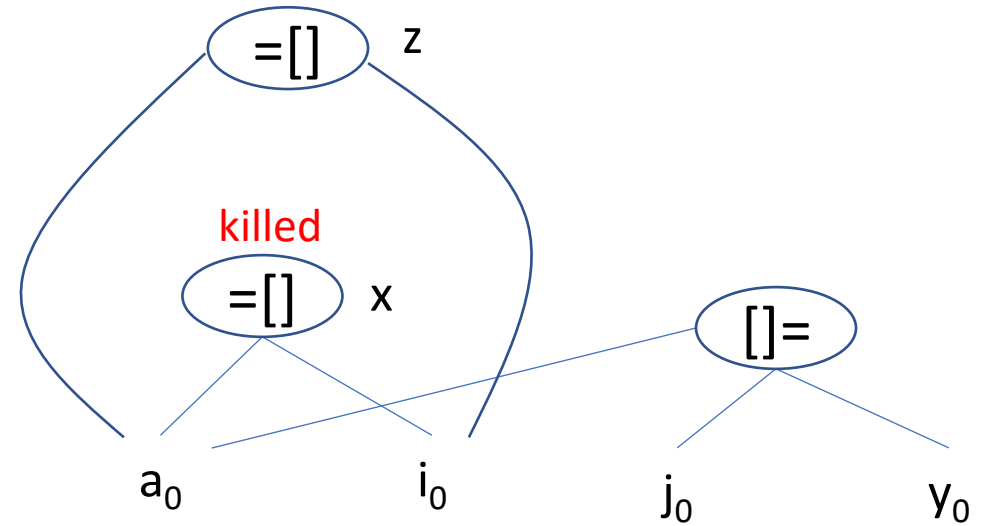
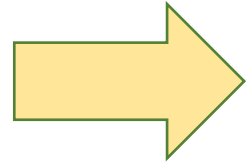


```
b = 12 + a  
x = b[i]  
b[j] = y
```

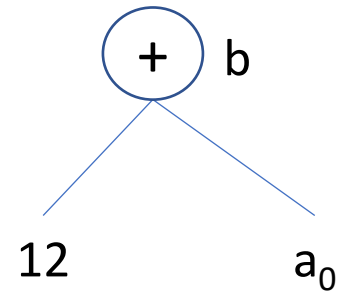
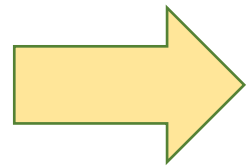


Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```

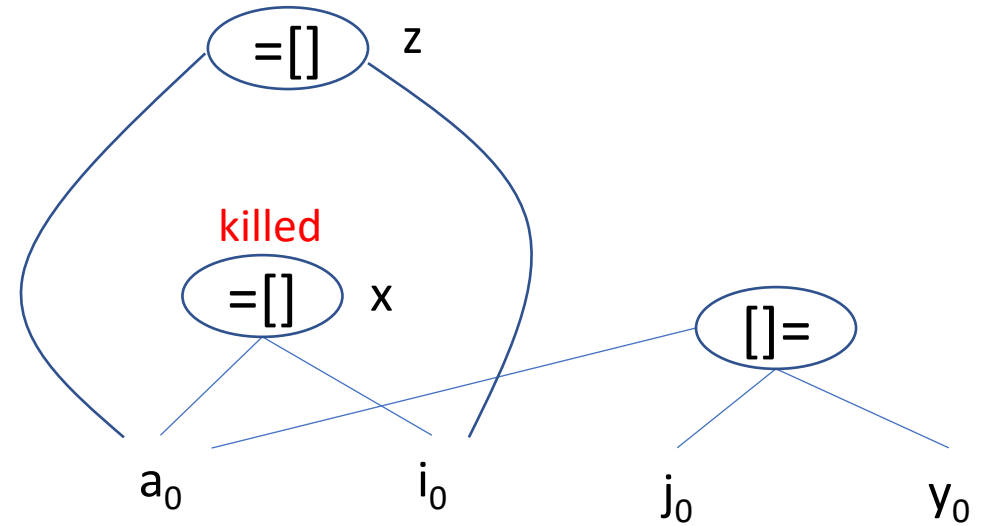
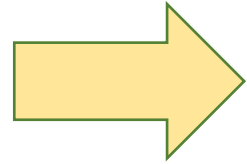


```
b = 12 + a  
x = b[i]  
b[j] = y
```

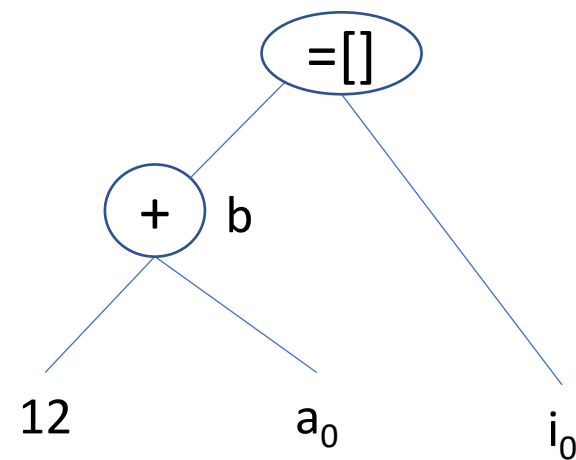
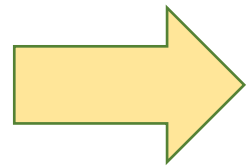


Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```

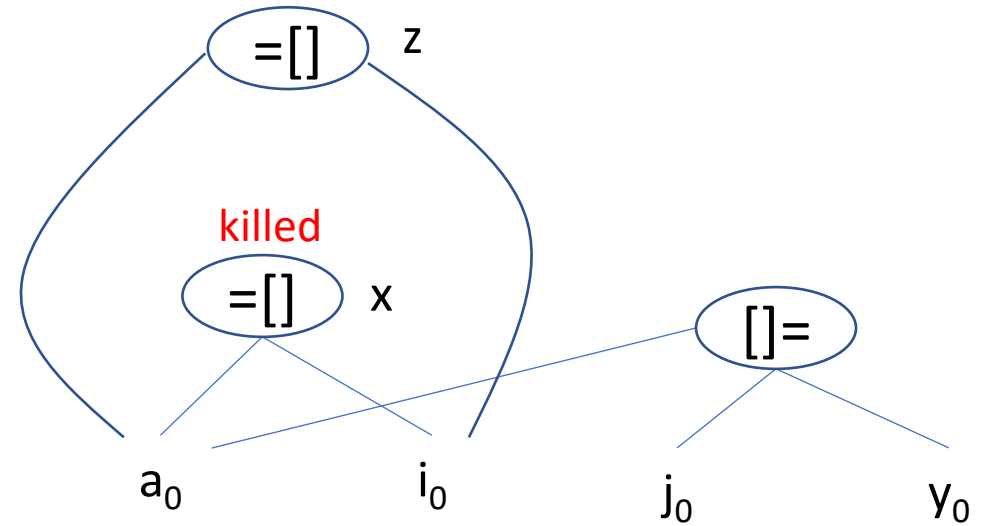
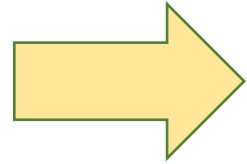


```
b = 12 + a  
x = b[i]  
b[j] = y
```

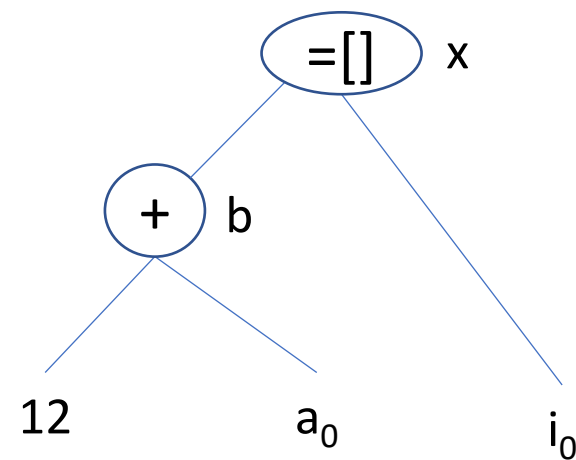
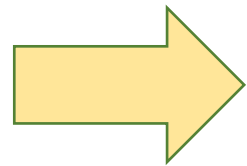


Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```

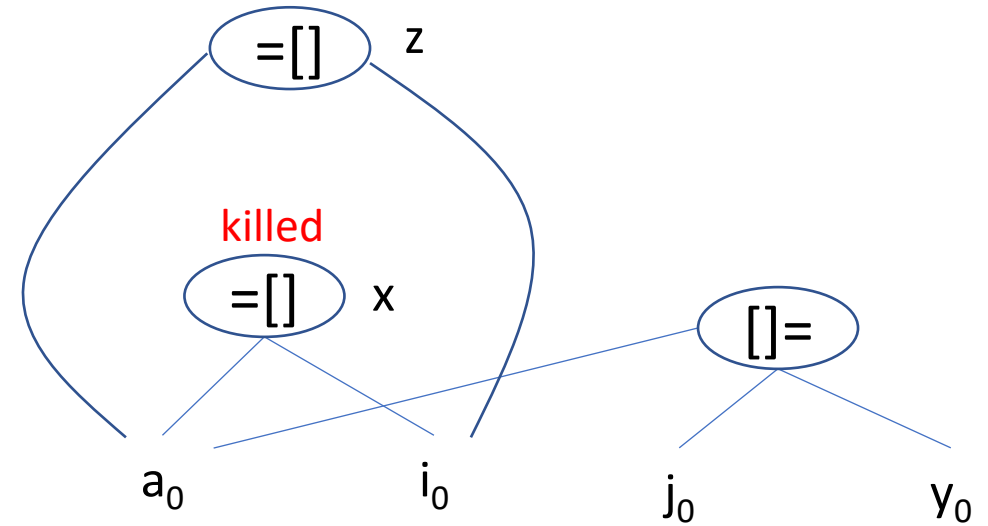
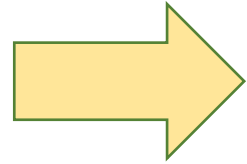


```
b = 12 + a  
x = b[i]  
b[j] = y
```

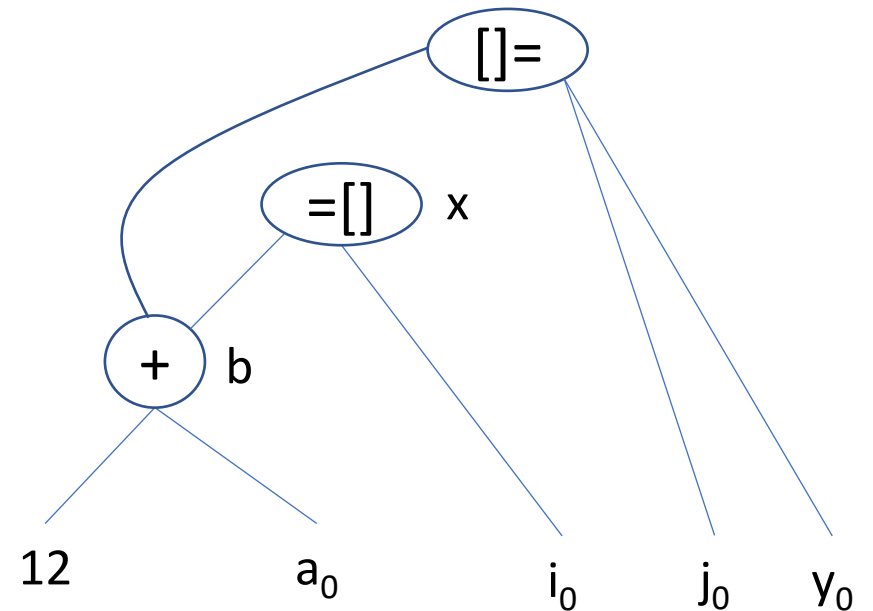
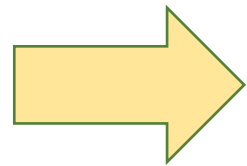


Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```

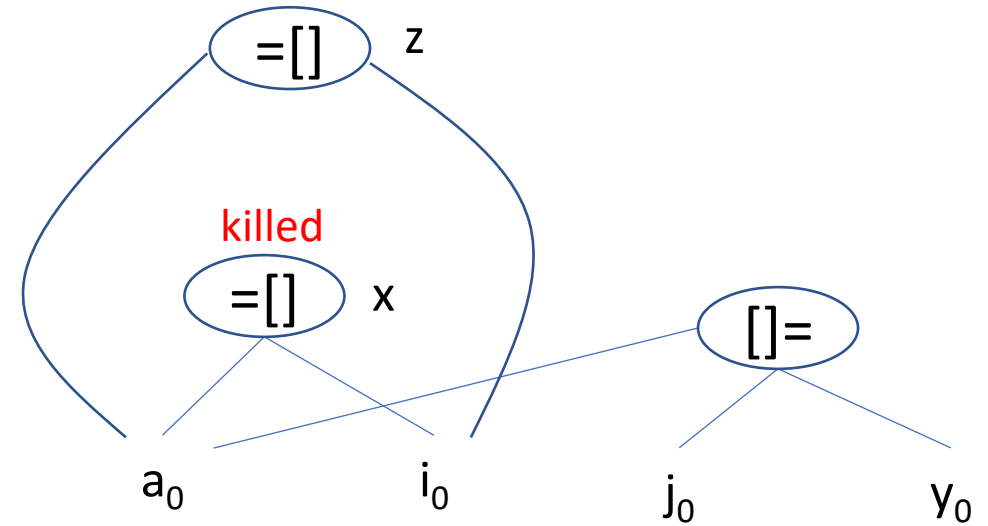
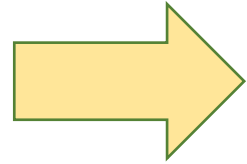


```
b = 12 + a  
x = b[i]  
b[j] = y
```

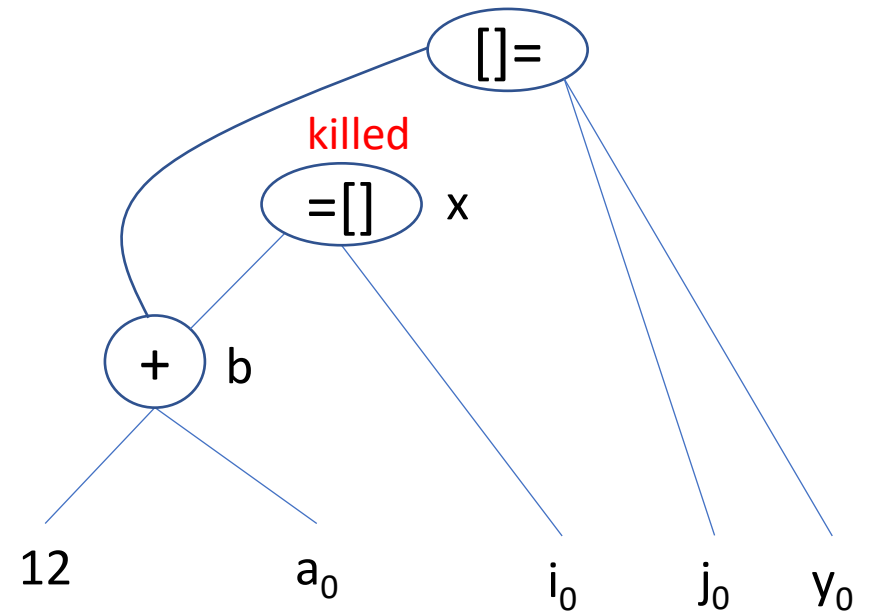
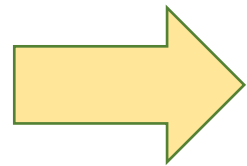


Representation of Array References

```
x = a[i]  
a[j] = y  
z = a[i]
```



```
b = 12 + a  
x = b[i]  
b[j] = y
```



Reassembling Basic Blocks From DAGs

Reassembling Basic Blocks From DAGs

- After performing optimizations on DAG, we need to reconstitute the three-address code for the basic block from which the DAG was built
- For each node, that has one or more attached variables, a three-address statement is constructed that computes the value of one of those variables
 - Select the variable that is live on exit from the block
 - However, if global-live variable information is not available, all variables except the temporaries are to be computed
- If a node has more than one live variable attached, then introduce copy statement to give the correct values to each of these variables.

Reassembling Basic Blocks From DAGs(2)

Reassembling Basic Blocks From DAGs(2)

- While reconstructing a basic block from the DAG, the order of instructions should also be correctly maintained conforming to the following rules -
 - A node's values cannot be computed until the value for each of its children has been computed
 - Assignments to an array must follow all previous assignments to, or evaluations from, the same array, according to the order of these instructions in the original basic block
 - Evaluations of array elements must follow any previous (according to the original block) assignments to the same array. The only permutation allowed is that two evaluations from the same array may be done in either order, as long as neither crosses over an assignment to that array
 - Any use of variable must follow all previous (according to the original block) procedure calls or indirect assignments through a pointer
 - Any procedure call or indirect assignments through a pointer must follow all previous (according to the original block) evaluation of any variable

Switch to
Machine Independent Optimizations
(Chapter 9)

Back to
Code Generation

A Simple Target Machine Model

A Simple Target Machine Model

- Target Language
 - **Assembly code** for a simple computer representative of many register machines
- Our target computer models –
 - A three - address machine
 - load/store operations
 - computational operations
 - jump/conditional jump operations
 - A byte addressable machine
 - Having n general purpose register
 - $R0, R1, \dots, Rn - 1$
 - All operands are integer

A Simple Target Machine Model(2)

A Simple Target Machine Model(2)

- Available Instructions:

- Load operations:

- **LD $dst, addr$**
 - Loads the value in location $addr$ into location dst
 - It denotes the assignment $dst = addr$
 - Examples –
 - **LD r, x** // loads value in location x into register r
 - **LD r_1, r_2** // register – to – register copy

- Store operations:

- **ST x, r**
 - Stores the value in register r into the location x
 - It denotes the assignment $x = r$

A Simple Target Machine Model

A Simple Target Machine Model

- Available Instructions:

- Computational operations:

- **OP** *dst, src₁, src₂*
 - OP is like ADD, SUB
 - SUB r_1, r_2, r_3 denotes $r_1 = r_2 - r_3$

- Unconditional jumps:

- **BR** *L*
 - Causes control to branch to the machine instruction with label *L*

A Simple Target Machine Model

A Simple Target Machine Model

- Available Instructions:

- Conditional jumps:

- **Bcond** r, L
 - Here r is a register
 - **cond** stands for a common tests on value in register r
 - Example:
 - **BLTZ** r, L
 - Jump to lable L if the value of r is less than 0

Addressing Modes

Addressing Modes

- Addressing Modes: from which location you can find the value for an operand

- **Absolute Mode**

- A location can be a variable name referring to the memory
- **LD r_1, x**

- **Register mode**

- A location can be a register
- **ST x, r_0**

Addressing Modes

Addressing Modes

- Addressing Modes: from which location you can find the value for an operand

➤ Indexed Mode

- A location can be an indexed address
- $a(r)$: a is a variable and r is a register.
 - Location is computed by adding the I – value of a to the value in register in r .
 - **LD R1, a(R2)**
 - $R1 = contents(a + contents(R2))$
- $c(r)$: here c is a constant and r is a register
 - **LD R1, 100(R2)**
 - $R1 = contents(100 + contents(R2))$

Addressing Modes

Addressing Modes

- Addressing Modes: from which location you can find the value for an operand

➤ Indirect modes:

- $*r$
 - LD R1, *R2
 - $R1 = \text{contents}(\text{contents}(R2))$
- $*a(r)$
 - LD R0, *x(R1)
 - $R0 = \text{contents}(\text{contents}(x + \text{contents}(R1)))$
- $*c(r)$
 - LD R0, *100(R1)
 - $R0 = \text{contents}(\text{contents}(100 + \text{contents}(R1)))$

Addressing Modes

Addressing Modes

- Addressing Modes: from which location you can find the value for an operand

➤ Immediate Constant Addressing Modes:

- Allow source to be a constant
- Constant prefixed by #
- LD R1, #100
 - $R1 = 100$
- ADD R1, R1, #100
 - $R1 = R1 + 100$

Statement-wise Code Generation

IR (TAC)

Target Code

Statement-wise Code Generation

IR (TAC)

`x = y - z`

Target Code

Statement-wise Code Generation

IR (TAC)

x = y - z

Target Code

```
LD  R1, y      // R1 = y
LD  R2, z      // R2 = z
SUB R1, R1, R2  // R1 = R1 - R2
ST  x, R1      // x = R1
```

Statement-wise Code Generation

IR (TAC)

`x = y - z`

`b = a[i]`

Target Code

```
LD  R1, y      // R1 = y
LD  R2, z      // R2 = z
SUB R1, R1, R2  // R1 = R1 - R2
ST  x, R1      // x = R1
```

Statement-wise Code Generation

IR (TAC)

`x = y - z`

`b = a[i]`

Target Code

```
LD  R1, y           // R1 = y
LD  R2, z           // R2 = z
SUB R1, R1, R2       // R1 = R1 - R2
ST  x, R1           // x = R1
```

```
LD  R1, i           // R1 = i
MUL R1, R1, 8        // R1 = R1 * 8
LD  R2, a(R1)        // R2 = contents(a + contents(R1))
ST  b, R2           // b = R2
```

Statement-wise Code Generation

IR (TAC)

`x = y - z`

`b = a[i]`

`a[j] = c`

Target Code

```
LD  R1, y           // R1 = y
LD  R2, z           // R2 = z
SUB R1, R1, R2       // R1 = R1 - R2
ST  x, R1           // x = R1
```

```
LD  R1, i           // R1 = i
MUL R1, R1, 8        // R1 = R1 * 8
LD  R2, a(R1)        // R2 = contents(a + contents(R1))
ST  b, R2           // b = R2
```

Statement-wise Code Generation

IR (TAC)

`x = y - z`

`b = a[i]`

`a[j] = c`

Target Code

```
LD  R1, y           // R1 = y
LD  R2, z           // R2 = z
SUB R1, R1, R2      // R1 = R1 - R2
ST  x, R1           // x = R1
```

```
LD  R1, i           // R1 = i
MUL R1, R1, 8       // R1 = R1 * 8
LD  R2, a(R1)       // R2 = contents(a + contents(R1))
ST  b, R2           // b = R2
```

```
LD  R1, c           // R1 = c
LD  R2, j           // R2 = j
MUL R2, R2, 8       // R2 = R2 * 8
ST  a(R2), R1       // contents(a + contents(R2)) = R1
```

Statement-wise Code Generation

IR (TAC)

Target Code

Statement-wise Code Generation

IR (TAC)

`x = *p`

Target Code

Statement-wise Code Generation

IR (TAC)

`x = *p`

Target Code

LD	R1, p	// R1 = p
LD	R2, 0(R1)	// R2 = contents(0 + contents(R1))
ST	x, R2	// x = R2

Statement-wise Code Generation

IR (TAC)

`x = *p`

`*p = y`

Target Code

```
LD  R1, p           // R1 = p
LD  R2, 0(R1)        // R2 = contents(0 + contents(R1))
ST  x, R2            // x = R2
```

Statement-wise Code Generation

IR (TAC)

`x = *p`

`*p = y`

Target Code

```
LD  R1, p           // R1 = p
LD  R2, 0(R1)        // R2 = contents(0 + contents(R1))
ST  x, R2            // x = R2
```

```
LD  R1, p           // R1 = p
LD  R2, y            // R2 = y
ST  0(R1), R2        // contents(0 + contents(R1)) = R2
```

Statement-wise Code Generation

IR (TAC)

`x = *p`

`*p = y`

`if x < y goto L`

Target Code

```
LD  R1, p           // R1 = p
LD  R2, 0(R1)        // R2 = contents(0 + contents(R1))
ST  x, R2            // x = R2
```

```
LD  R1, p           // R1 = p
LD  R2, y            // R2 = y
ST  0(R1), R2        // contents(0 + contents(R1)) = R2
```

Statement-wise Code Generation

IR (TAC)

`x = *p`

`*p = y`

`if x < y goto L`

Target Code

```
LD  R1, p           // R1 = p
LD  R2, 0(R1)        // R2 = contents(0 + contents(R1))
ST  x, R2            // x = R2
```

```
LD  R1, p           // R1 = p
LD  R2, y            // R2 = y
ST  0(R1), R2        // contents(0 + contents(R1)) = R2
```

```
LD  R1, x           // R1 = x
LD  R2, y            // R2 = y
SUB  R1, R1, R2       // R1 = R1 - R2
BLTZ R1, M           // if R1 < 0 jump to M
```

Program Cost

Program Cost

- A cost is often associated with compiling and running program
- Common cost measure may be-
 - Compilation time
 - Size of the target program
 - Running-time of the target program
 - Power consumption of the target program
- We assume, the cost of a target-language program on a given input is the sum of the costs of the individual instructions executed when the program is run on that input.

Instruction Cost

Instruction Cost

- Simple model for instruction cost
 - Cost of an instruction = One plus cost for the addressing modes.
- Addressing modes –
 - involving registers have zero additional cost
 - involving a memory location / constant have an additional cost
 - Since such operands have to be stored in the words following the instruction

Instruction Cost

Instruction Cost

- Examples:

1. **LD R0, R1**

- Meaning: $R0 = \text{contents}(R1)$
- Cost = $1 + 0 = 1$

2. **LD R0, M**

- Meaning: $R0 = \text{contents}(M)$
- Cost = $1 + 1 = 1$

3. **LD R1, * 100(R2)**

- Meaning: $R1 = \text{contents}(\text{contents}(100 + \text{contents}(R2)))$
- Cost = $1 + 2 = 3$

4. **ADD R1, #3, #4**

- Meaning: $R1 = 3 + 4$
- Cost = $1 + 2 = 3$

Uses of Registers

Uses of Registers

- One of the primary issues during code generation is deciding how to use registers to best advantage
- There are four principal uses of registers
 - In most machine architectures, some or all of the operands of an operation must be in registers
 - Registers make good temporaries –
 - Hold the result of a subexpression while evaluating a larger expression
 - Hold a variable that is used only within a single basic block
 - Registers are used to hold (*global*) values that are computed in one basic block and used in other blocks, for example a **loop index**
 - Registers are often used to help with run-time storage management, for example maintaining **stack pointers** and **top of the stack**

Simple Code-Generation Algorithm

Simple Code-Generation Algorithm

- Generates code for a single basic block
- **Assumes** -
 - the basic block has already been optimized
 - some set of registers are available to hold values within blocks
 - there is exactly one machine instruction for each **operator** that accepts the operands from registers and stores the result in a registers
- Consider each three-address instruction in turn
- Keep track of what values are in what registers
 - Avoid generating unnecessary loads and stores

Register and Address Descriptors

Register and Address Descriptors

- **Register Descriptor**

- One for each register
- Keeps track of the variable names whose current value is in the register
- Initially all are set to empty

- **Address Descriptor**

- One for each program variable
- Keeps track of the location or locations where the current value of that variable can be found
 - Locations can be a register, a memory address, a stack location
 - Can be a set of more than one of the above
 - Can be stored in the symbol table entry of the variable

An Example of Code-Generation

IR (TAC)

Gen. Code

getReg(l)

Register Descriptors

Address Descriptors

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

R1	R2	R3	a	b	c	d	t	u	v
			a	b	c	d			

An Example of Code-Generation

IR (TAC)

Gen. Code

getReg(l)

Register Descriptors

Address Descriptors

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

R1	R2	R3	a	b	c	d	t	u	v
			a	b	c	d			

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

getReg(l)

{R2, R1, R2}

Register Descriptors

R1	R2	R3

Address Descriptors

a	b	c	d	t	u	v
a	b	c	d			

An Example of Code-Generation

IR (TAC)

Gen. Code

getReg(I)

Register Descriptors

Address Descriptors

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```



```
LD R1, a
LD R2, b
SUB R2, R1, R2
```

{R2, R1, R2}

R1	R2	R3

a	b	c	d	t	u	v
a	b	c	d			

An Example of Code-Generation

IR (TAC)

Gen. Code

getReg(I)

Register Descriptors

Address Descriptors

`t = a - b`
`u = a - c`
`v = t + u`
`a = d`
`d = v + u`



`LD R1, a`
`LD R2, b`
`SUB R2, R1, R2`

`{R2, R1, R2}`

R1	R2	R3

a	t	
---	---	--

a	b	c	d	t	u	v
a	b	c	d			

An Example of Code-Generation

IR (TAC)

Gen. Code

getReg(I)

Register Descriptors

Address Descriptors

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```



```
LD R1, a
LD R2, b
SUB R2, R1, R2
```

{R2, R1, R2}

R1	R2	R3

a	b	c	d	t	u	v
a	b	c	d			

a	t	
---	---	--

a, R1	b	c	d	R2		
-------	---	---	---	----	--	--

An Example of Code-Generation

IR (TAC)

Gen. Code

getReg(I)

Register Descriptors

Address Descriptors

$t = a - b$
 $u = a - c$
 $v = t + u$
 $a = d$
 $d = v + u$

$\rightarrow$ LD R1, a
 LD R2, b
 SUB R2, R1, R2

{R2, R1, R2}

R1	R2	R3

a	b	c	d	t	u	v
a	b	c	d			

a	t	
---	---	--

a, R1	b	c	d	R2		
-------	---	---	---	----	--	--

An Example of Code-Generation

IR (TAC)

Gen. Code

getReg(l)

Register Descriptors

Address Descriptors

```

t = a - b
u = a - c
v = t + u
a = d
d = v + u
    
```

LD R1, a
LD R2, b
SUB R2, R1, R2

{R2, R1, R2}

{R1, R1, R3}

R1	R2	R3

a	b	c	d	t	u	v
a	b	c	d			

a	t	
a	t	

a, R1	b	c	d	R2		
a, R1	b	c	d	R2		

An Example of Code-Generation

IR (TAC)

Gen. Code

getReg(I)

Register Descriptors

Address Descriptors

t = a - b
u = a - c
v = t + u
a = d
d = v + u

LD R1, a
LD R2, b
SUB R2, R1, R2

LD R3, c
SUB R1, R1, R3

{R2, R1, R2}

{R1, R1, R3}

R1	R2	R3

a	t	
---	---	--

a	b	c	d	t	u	v
a	b	c	d			

a, R1	b	c	d	R2		
-------	---	---	---	----	--	--

An Example of Code-Generation

IR (TAC)

Gen. Code

getReg(I)

Register Descriptors

Address Descriptors

```

t = a - b
u = a - c
v = t + u
a = d
d = v + u
    
```

→ LD R1, a
 LD R2, b
 SUB R2, R1, R2

→ LD R3, c
 SUB R1, R1, R3

{R2, R1, R2}

{R1, R1, R3}

R1	R2	R3

a	t	
---	---	--

u	t	c
---	---	---

a	b	c	d	t	u	v
a	b	c	d			

a, R1	b	c	d	R2		
-------	---	---	---	----	--	--

An Example of Code-Generation

IR (TAC)

Gen. Code

getReg(I)

Register Descriptors

Address Descriptors

```

t = a - b
u = a - c
v = t + u
a = d
d = v + u
    
```

→ LD R1, a
 LD R2, b
 SUB R2, R1, R2

→ LD R3, c
 SUB R1, R1, R3

{R2, R1, R2}

{R1, R1, R3}

R1	R2	R3

a	b	c	d	t	u	v
a	b	c	d			

a	t	
---	---	--

a, R1	b	c	d	R2		
-------	---	---	---	----	--	--

u	t	c
---	---	---

a	b	c, R3	d	R2	R1	
---	---	-------	---	----	----	--

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

getReg(l)

Register Descriptors

Address Descriptors

R1	R2	R3	a	b	c	d	t	u	v
u	t	c	a	b	c, R3	d	R2	R1	

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

getReg(l)

Register Descriptors

Address Descriptors

R1	R2	R3	a	b	c	d	t	u	v
u	t	c	a	b	c, R3	d	R2	R1	

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

getReg(l)

{R3, R2, R1}

Register Descriptors

R1	R2	R3
u	t	c

Address Descriptors

a	b	c	d	t	u	v
a	b	c, R3	d	R2	R1	

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

ADD R3, R2, R1

getReg(l)

{R3, R2, R1}

Register Descriptors

R1	R2	R3
u	t	c

Address Descriptors

a	b	c	d	t	u	v
a	b	c, R3	d	R2	R1	

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

ADD R3, R2, R1

getReg(l)

{R3, R2, R1}

Register Descriptors

R1	R2	R3
u	t	c

u	t	v
---	---	---

Address Descriptors

a	b	c	d	t	u	v
a	b	c, R3	d	R2	R1	

a	b	c	d	R2	R1	R3
---	---	---	---	----	----	----

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

ADD R3, R2, R1

getReg(l)

{R3, R2, R1}

Register Descriptors

R1	R2	R3	a	b	c	d	t	u	v
u	t	c	a	b	c, R3	d	R2	R1	
u	t	v	a	b	c	d	R2	R1	R3

Address Descriptors

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

ADD R3, R2, R1

getReg(l)

{R3, R2, R1}

{R2, R2}

Register Descriptors

R1	R2	R3
u	t	c

u	t	v
---	---	---

Address Descriptors

a	b	c	d	t	u	v
a	b	c, R3	d	R2	R1	

a	b	c	d	R2	R1	R3
---	---	---	---	----	----	----

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

ADD R3, R2, R1

LD R2, d

getReg(l)

{R3, R2, R1}

{R2, R2}

Register Descriptors

R1	R2	R3
u	t	c

R1	R2	R3
u	t	v

Address Descriptors

a	b	c	d	t	u	v
a	b	c, R3	d	R2	R1	

a	b	c	d	t	u	v
a	b	c	d	R2	R1	R3

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

ADD R3, R2, R1

LD R2, d

getReg(l)

{R3, R2, R1}

{R2, R2}

Register Descriptors

R1	R2	R3	a	b	c	d	t	u	v
u	t	c	a	b	c, R3	d	R2	R1	
u	t	v	a	b	c	d	R2	R1	R3
u	a, d	v	R2	b	c	d, R2		R1	R3

Address Descriptors

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

ADD R3, R2, R1

LD R2, d

getReg(l)

{R3, R2, R1}

{R2, R2}

Register Descriptors

R1	R2	R3	a	b	c	d	t	u	v
u	t	c	a	b	c, R3	d	R2	R1	
u	t	v	a	b	c	d	R2	R1	R3
u	a, d	v	R2	b	c	d, R2		R1	R3

Address Descriptors

An Example of Code-Generation

IR (TAC)

```
t = a - b
u = a - c
v = t + u
a = d
d = v + u
```

Gen. Code

ADD R3, R2, R1

LD R2, d

getReg(l)

{R3, R2, R1}

{R2, R2}

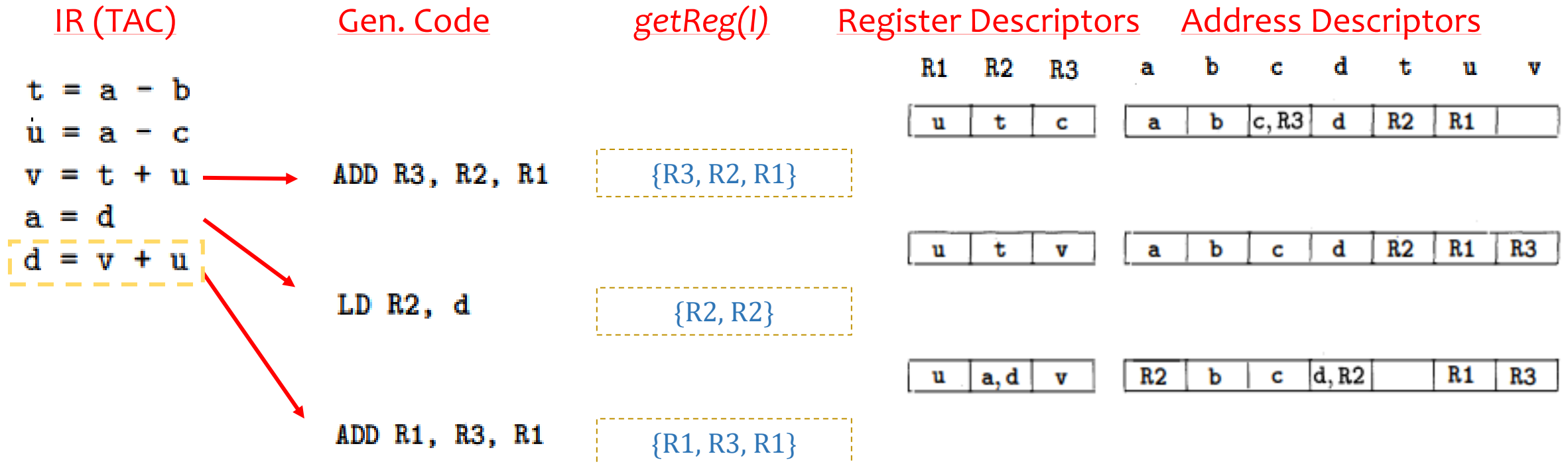
{R1, R3, R1}

Register Descriptors

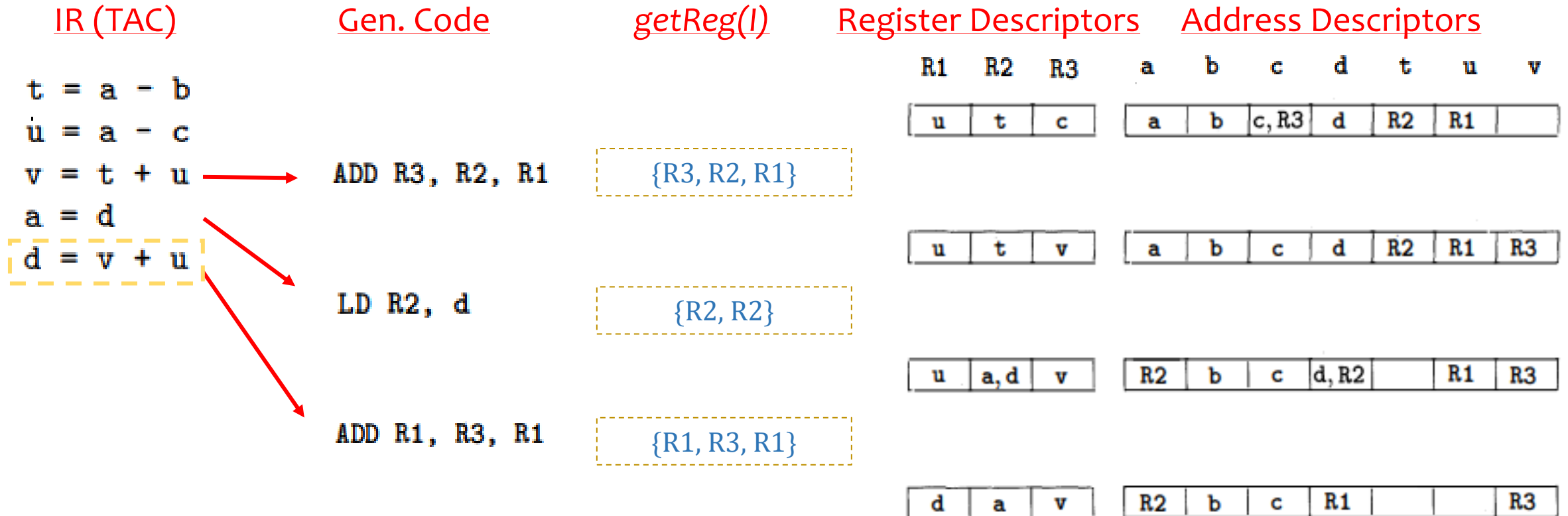
R1	R2	R3	a	b	c	d	t	u	v
u	t	c	a	b	c, R3	d	R2	R1	
u	t	v	a	b	c	d	R2	R1	R3
u	a, d	v	R2	b	c	d, R2		R1	R3

Address Descriptors

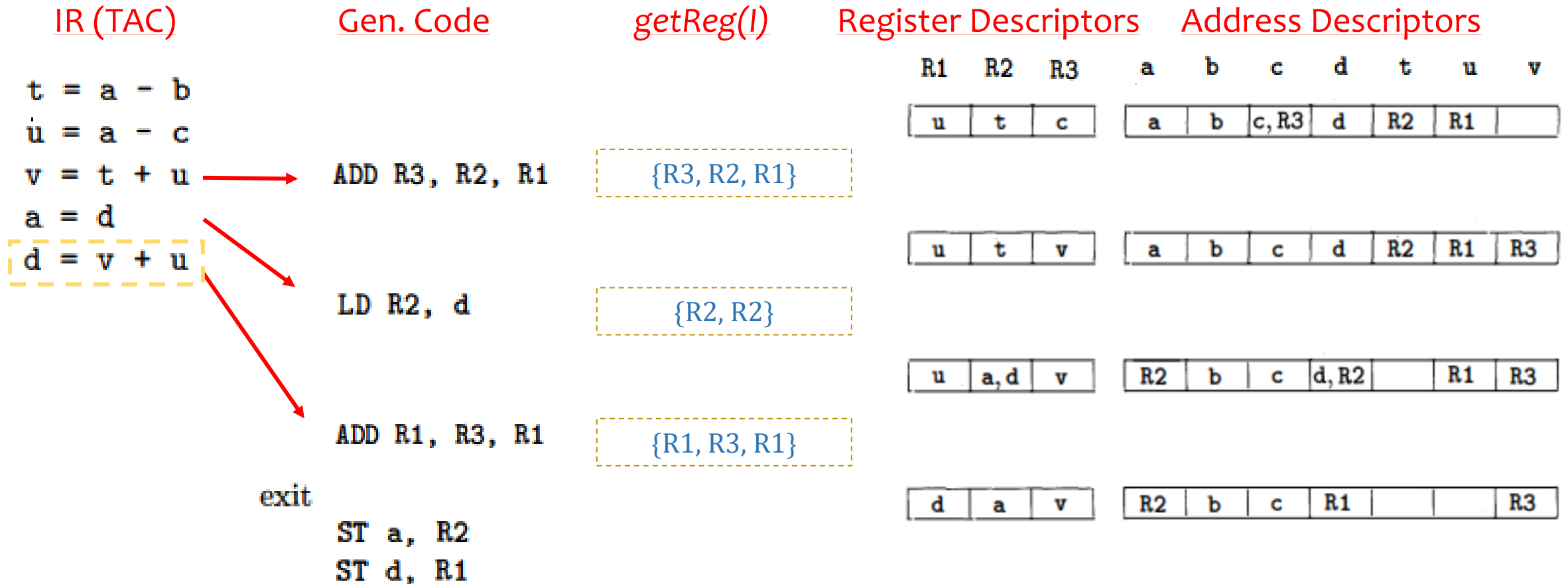
An Example of Code-Generation



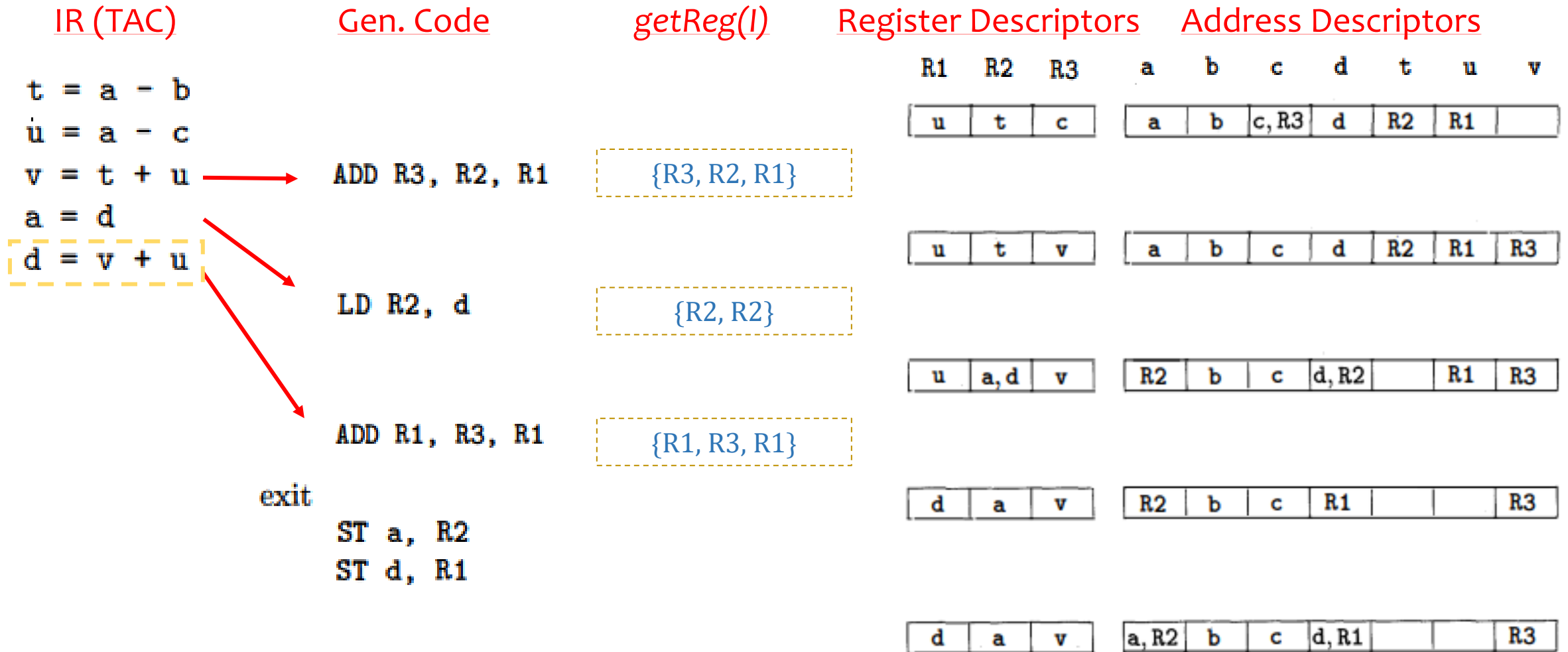
An Example of Code-Generation



An Example of Code-Generation



An Example of Code-Generation



The Code-Generation Algorithm

- Machine instructions for **operations**

The Code-Generation Algorithm

- Machine instructions for **operations**

For a three-address instruction such as $x = y + z$, do the following:

1. Use $getReg(x = y + z)$ to select registers for x , y , and z . Call these R_x , R_y , and R_z .
2. If y is not in R_y (according to the register descriptor for R_y), then issue an instruction $LD\ R_y, y'$, where y' is one of the memory locations for y (according to the address descriptor for y).
3. Similarly, if z is not in R_z , issue an instruction $LD\ R_z, z'$, where z' is a location for z .
4. Issue the instruction $ADD\ R_x, R_y, R_z$.

The Code-Generation Algorithm

The Code-Generation Algorithm

- Machine instructions for **Copy Statements** (of the form $x=y$)
 - Assumes *getReg(I)* will always generate the same register for x and y
 - If y is not already in R_y , then generate instruction **LD R_y, y**
 - Else do nothing
- Ending the basic block
 - Variables used in a block may wind up with their only location being a register
 - For temporary variables within a block, do nothing
 - For each variable x that is live on exit from the block, generate instruction **ST x, R** where R is the register that stores x 's value

Managing Register and Address Descriptors

Managing Register and Address Descriptors

1. For the instruction `LD  $R$ ,  $x$`
 - (a) Change the register descriptor for register R so it holds only x .
 - (b) Change the address descriptor for x by adding register R as an additional location.
2. For the instruction `ST  $x$ ,  $R$` , change the address descriptor for x to include its own memory location.

Managing Register and Address Descriptors

Managing Register and Address Descriptors

3. For an operation such as `ADD  $R_x, R_y, R_z$` implementing a three-address instruction $x = y + z$
 - (a) Change the register descriptor for R_x so that it holds only x .
 - (b) Change the address descriptor for x so that its only location is R_x .
Note that the memory location for x is *not* now in the address descriptor for x .
 - (c) Remove R_x from the address descriptor of any variable other than x .
4. When we process a copy statement $x = y$, after generating the load for y into register R_y , if needed, and after managing descriptors as for all load statements (per rule 1):
 - (a) Add x to the register descriptor for R_y .
 - (b) Change the address descriptor for x so that its only location is R_y .

Design of Function getReg(*l*)

Design of Function getReg(*l*)

- For variable *y* (and *z* as well) in the expression $x = y + z$
 - If *y* is currently in a register, pick that register as R_y
 - Else if *y* is not in a register and there are **empty registers**, pick an empty register as R_y
 - Else if *y* is not in a register and there are **no empty registers**, then
 - for each register R
 - For each variable *v* that are in R
 - If *v* is somewhere besides R (check from address descriptor), pick R
 - Else if *v* is *x*, and *x* is not one of the other operands (*z*), pick R
 - Else if *v* is not used later, pick R
 - Else generate a store instruction **ST *v*, R** ; called a *spill*
 - Since R may hold several variables at the moment, calculate R 's score as total number of spills required for all its variables
 - pick the register R with lowest score as R_y

Design of Function getReg(*l*)

Design of Function getReg(*I*)

- For variable *x* in the expression $x = y + z$
 - The issues and options are almost as for *y*, except for the following differences
 - A register that holds only *x* is always an acceptable choice for R_x
 - holds true even if *x* is one of *y* and *z*
 - If *y* is not used after instruction *I* and R_y holds only *y* after being loaded, if necessary then pick R_y as R_x
 - similar option holds regarding *z* and R_z
 - If *I* is a copy instruction ($x = y$), always choose R_y as R_x

Global Register Allocation

(See the pdf titled Global Register Allocation)

Related References

Related References

- <https://web.stanford.edu/class/cs143/lectures/lecture17.pdf>
- <http://web.cs.ucla.edu/~palsberg/course/cs132/linearscan.pdf>
- <https://user.it.uu.se/~kostis/Teaching/KT1-11/Slides/lecture16.pdf>
- https://en.wikipedia.org/wiki/Register_allocation

An Example From GCC

Source Code

```
int a, b, c, d,  
e, f, g;  
int main() {  
    a=b*c;  
    b=1;  
    return 0;  
}
```

An Example From GCC

Source Code

```
int a, b, c, d,  
e, f, g;  
int main() {  
    a=b*c;  
    b=1;  
    return 0;  
}
```

Target Code (MIPS64 gcc 5.4 without O1 switch)

```
... ..  
ld $2,%got_disp(b)($4)  
lw $3,0($2)  
ld $2,%got_disp(c)($4)  
lw $2,0($2)  
mult $3,$2  
mflo $3  
ld $2,%got_disp(a)($4)  
sw $3,0($2)  
ld $2,%got_disp(b)($4)  
li $3,1 # 0x1  
sw $3,0($2)  
... ..
```

An Example From GCC

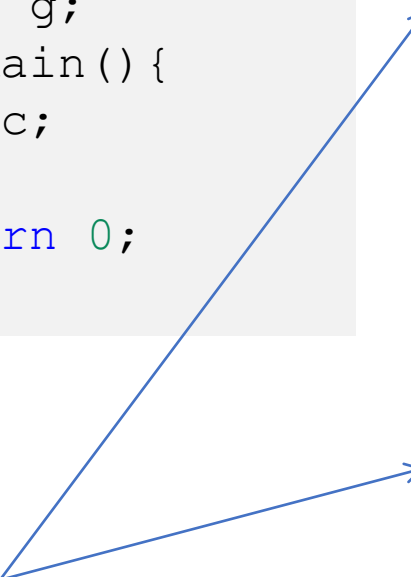
Source Code

```
int a, b, c, d,  
e, f, g;  
int main() {  
    a=b*c;  
    b=1;  
    return 0;  
}
```

Target Code (MIPS64 gcc 5.4 without O1 switch)

```
... ..  
ld $2,%got_disp(b)($4)  
lw $3,0($2)  
ld $2,%got_disp(c)($4)  
lw $2,0($2)  
mult $3,$2  
mflo $3  
ld $2,%got_disp(a)($4)  
sw $3,0($2)  
ld $2,%got_disp(b)($4)  
li $3,1 # 0x1  
sw $3,0($2)  
... ..
```

Multiple loads of
variable b



An Example From GCC

Source Code

```
int a, b, c, d,  
e, f, g;  
int main() {  
    a=b*c;  
    b=1;  
    return 0;  
}
```

Target Code (MIPS64 gcc 5.4 without O1 switch)

```
... ..  
ld $2,%got_disp(b)($4)  
lw $3,0($2)  
ld $2,%got_disp(c)($4)  
lw $2,0($2)  
mult $3,$2  
mflo $3  
ld $2,%got_disp(a)($4)  
sw $3,0($2)  
ld $2,%got_disp(b)($4)  
li $3,1 # 0x1  
sw $3,0($2)  
... ..
```

Multiple loads of
variable b

Target Code (MIPS64 gcc 5.4 with O1 switch)

```
... ..  
ld $3,%got_disp(b)($5)  
lw $2,0($3)  
ld $4,%got_disp(c)($5)  
lw $4,0($4)  
mul $2,$2,$4  
ld $4,%got_disp(a)($5)  
sw $2,0($4)  
li $2,1 # 0x1  
sw $2,0($3)  
... ..
```

All variables are
loaded only once

Peephole Optimization

- **Peephole**
 - a small sliding window of target instructions
- Replace instructions in peephole with shorter and faster sequence
- Can be applied on Intermediate code or assembly code
- Repeated passes result in better optimization

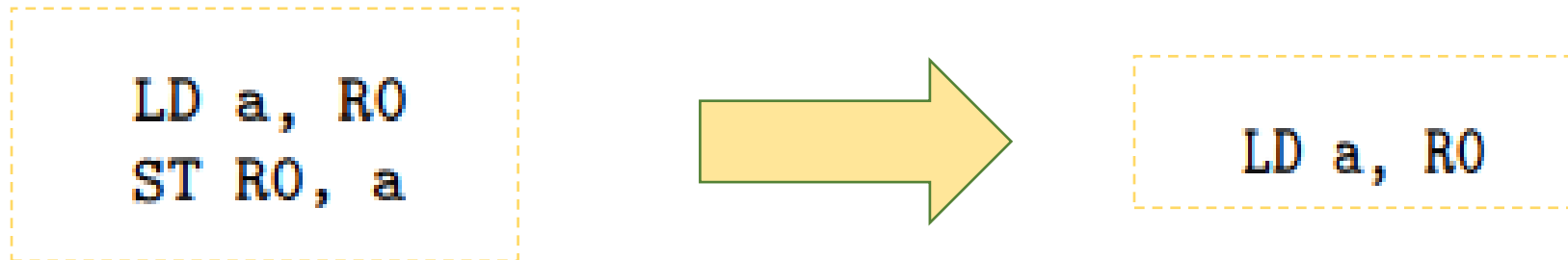
Peephole Optimization

- Redundant-instruction elimination

```
LD a, R0  
ST R0, a
```

Peephole Optimization

- Redundant-instruction elimination



Peephole Optimization

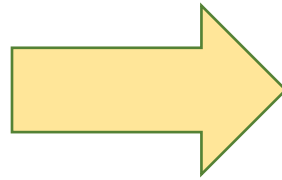
- Eliminating unreachable code

```
    if debug == 1 goto L1  
    goto L2  
L1: print debugging information  
L2:
```

Peephole Optimization

- Eliminating unreachable code

```
    if debug == 1 goto L1  
    goto L2  
L1: print debugging information  
L2:
```



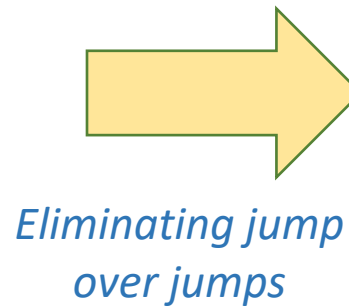
*Eliminating jump
over jumps*

```
    if debug != 1 goto L2  
    print debugging information  
L2:
```

Peephole Optimization

- Eliminating unreachable code

```
if debug == 1 goto L1  
goto L2  
L1: print debugging information  
L2:
```



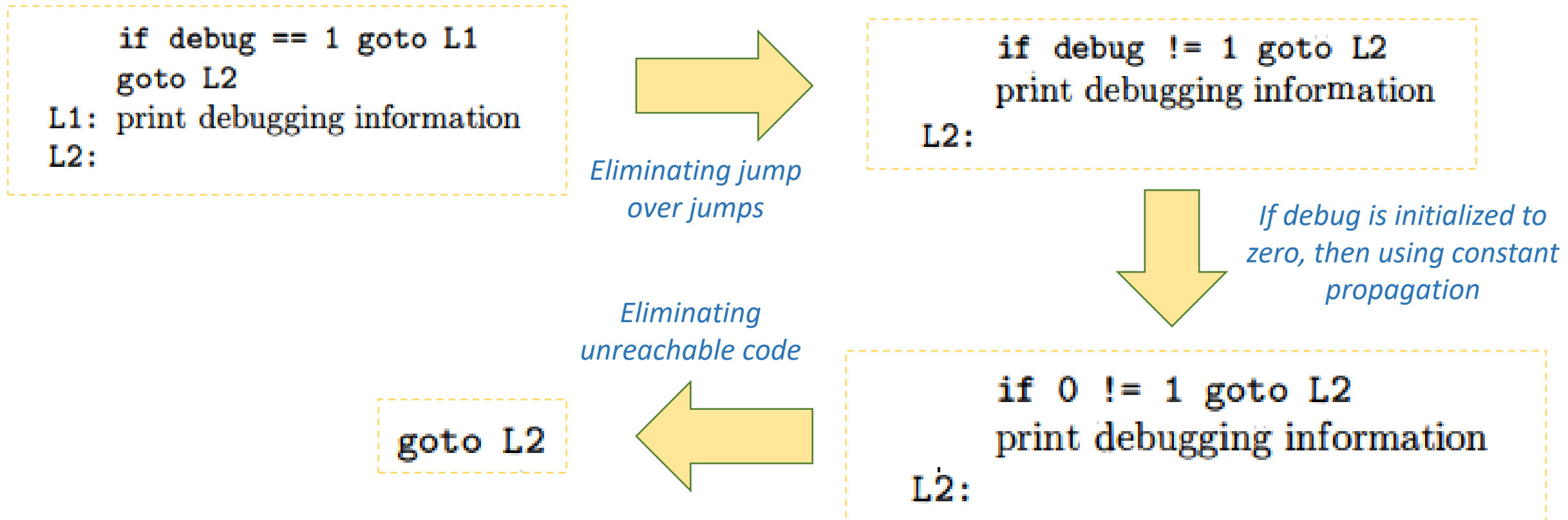
```
if debug != 1 goto L2  
print debugging information  
L2:
```



```
if 0 != 1 goto L2  
print debugging information  
L2:
```

Peephole Optimization

- Eliminating unreachable code



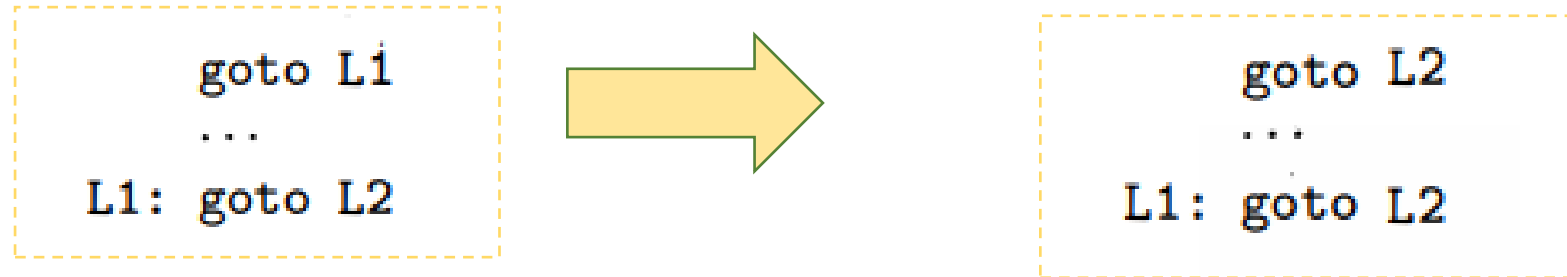
Peephole Optimization

- Flow-of-control optimization

```
    goto L1  
    ...  
L1: goto L2
```

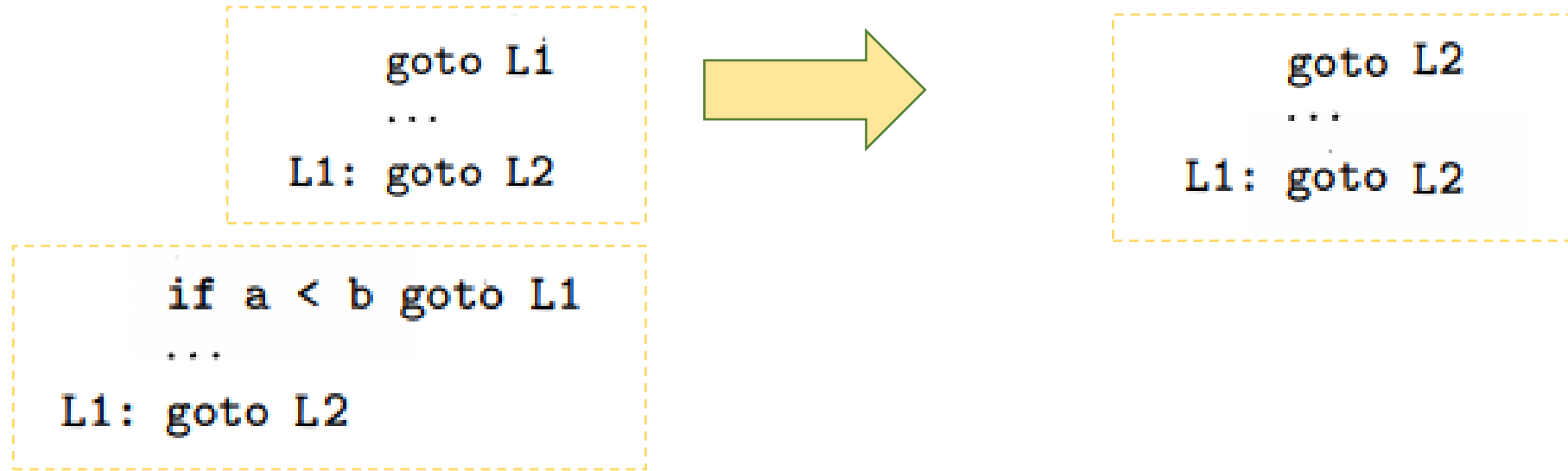
Peephole Optimization

- Flow-of-control optimization



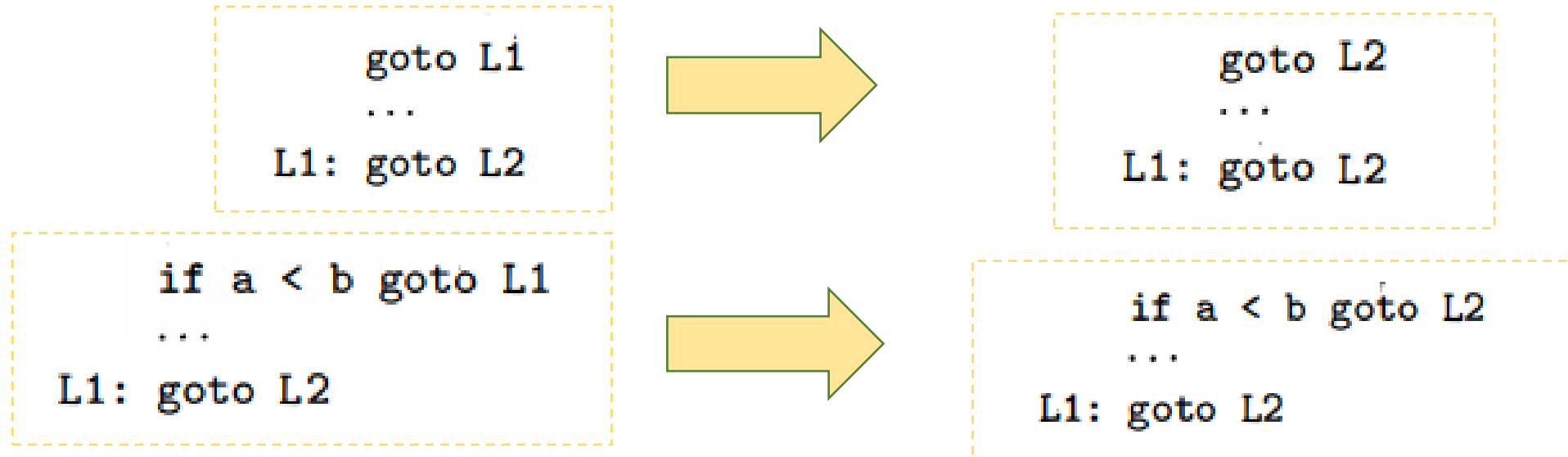
Peephole Optimization

- Flow-of-control optimization



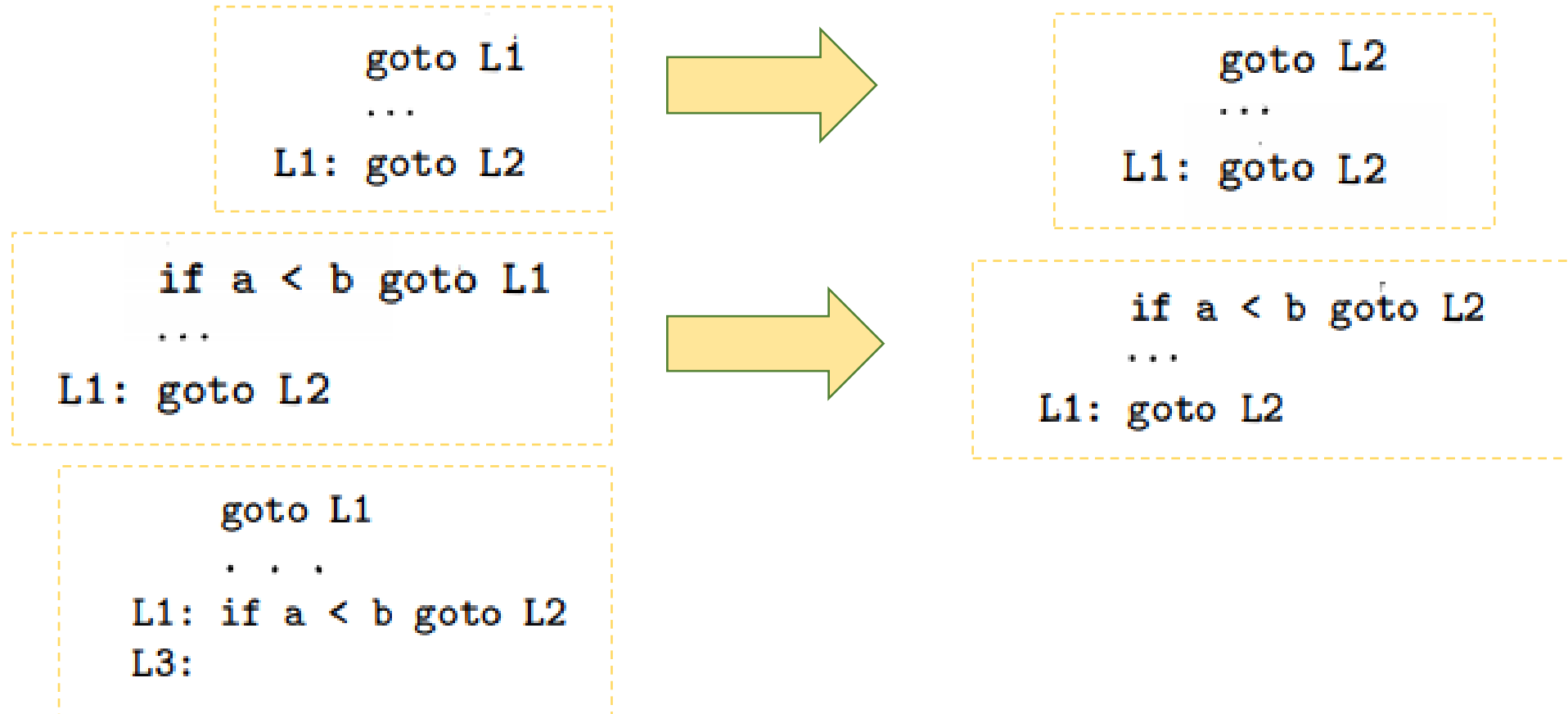
Peephole Optimization

- Flow-of-control optimization



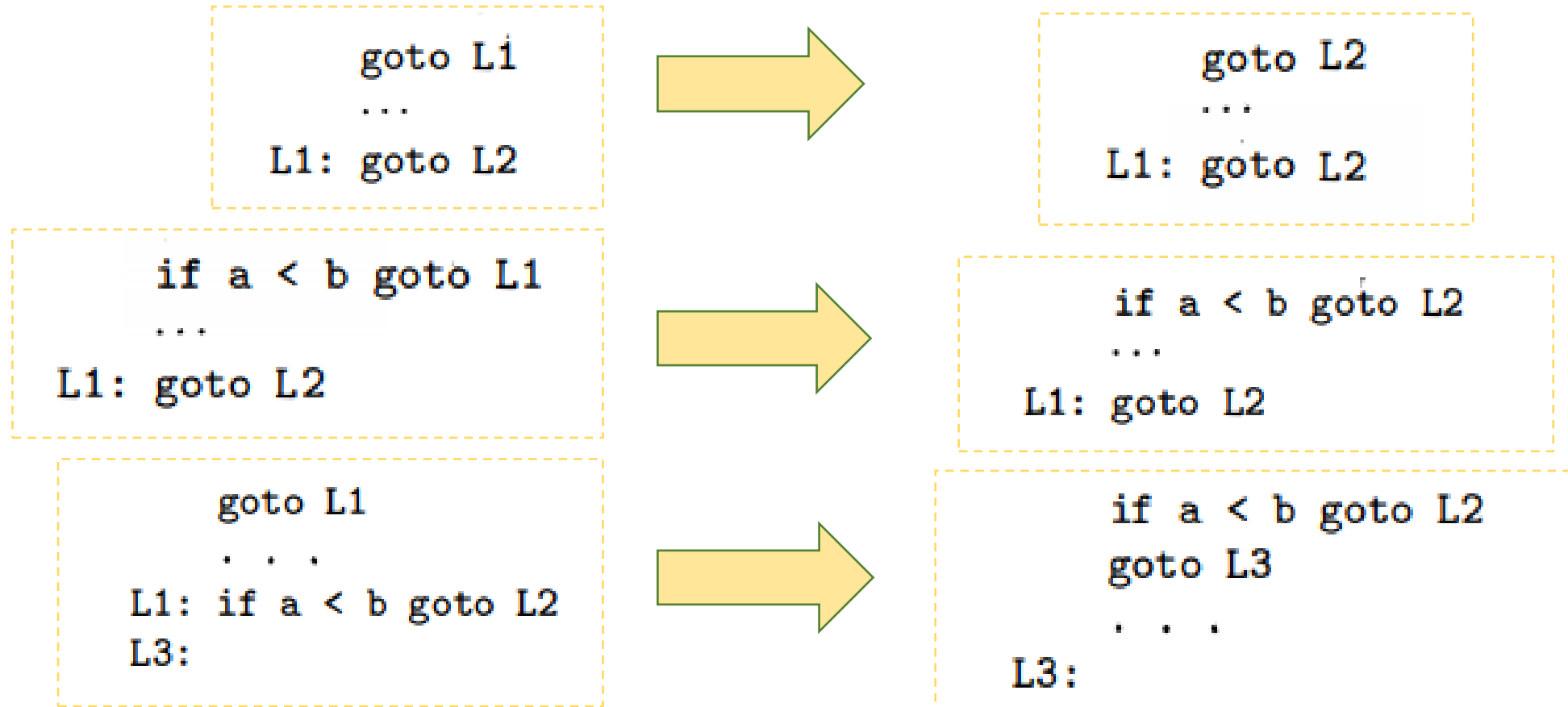
Peephole Optimization

- Flow-of-control optimization



Peephole Optimization

- Flow-of-control optimization



Peephole Optimization

- Algebraic Simplification
- Reduction in strength

Peephole Optimization

- Algebraic Simplification

$$x + 0 = 0 + x = x$$

$$x \times 1 = 1 \times x = x$$

$$x - 0 = x$$

$$x/1 = x$$

- Reduction in strength

Peephole Optimization

- Algebraic Simplification

$$x + 0 = 0 + x = x$$

$$x \times 1 = 1 \times x = x$$

$$x - 0 = x$$

$$x/1 = x$$

- Reduction in strength

EXPENSIVE

CHEAPER

$$x^2$$

=

$$x \times x$$

$$2 \times x$$

=

$$x + x$$

$$x/2$$

=

$$x \times 0.5$$

Peephole Optimization

- Use of machine idioms



Reference

- Compilers: Principles, Techniques, & Tools, by Alfred B. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman
 - Chapter 8
 - 8.1
 - 8.2
 - 8.3
 - 8.4
 - 8.5
 - 8.6
 - 8.7

Thank You!